



المحاضرة السادسة

التوابع (الدوال)

Functions

Topics for Lecture 6

- 6.1 Focus on Software Engineering: Modular Programming
- 6.2 Defining and Calling Functions
- 6.3 Function Prototypes
- 6.4 Sending Data into a Function
- 6.5 Passing Data by Value
- 6.6 Focus on Software Engineering: Using Functions in a Menu-Driven Program
- 6.7 The return Statement
- 6.8 Returning a Value from a Function
- 6.9 Returning a Boolean Value
- 6.10 Local and Global Variables
- 6.11 Static Local Variables
- 6.12 Default Arguments
- 6.13 Using Reference Variables as Parameters
- 6.14 Overloading Functions
- 6.15 The exit() Function

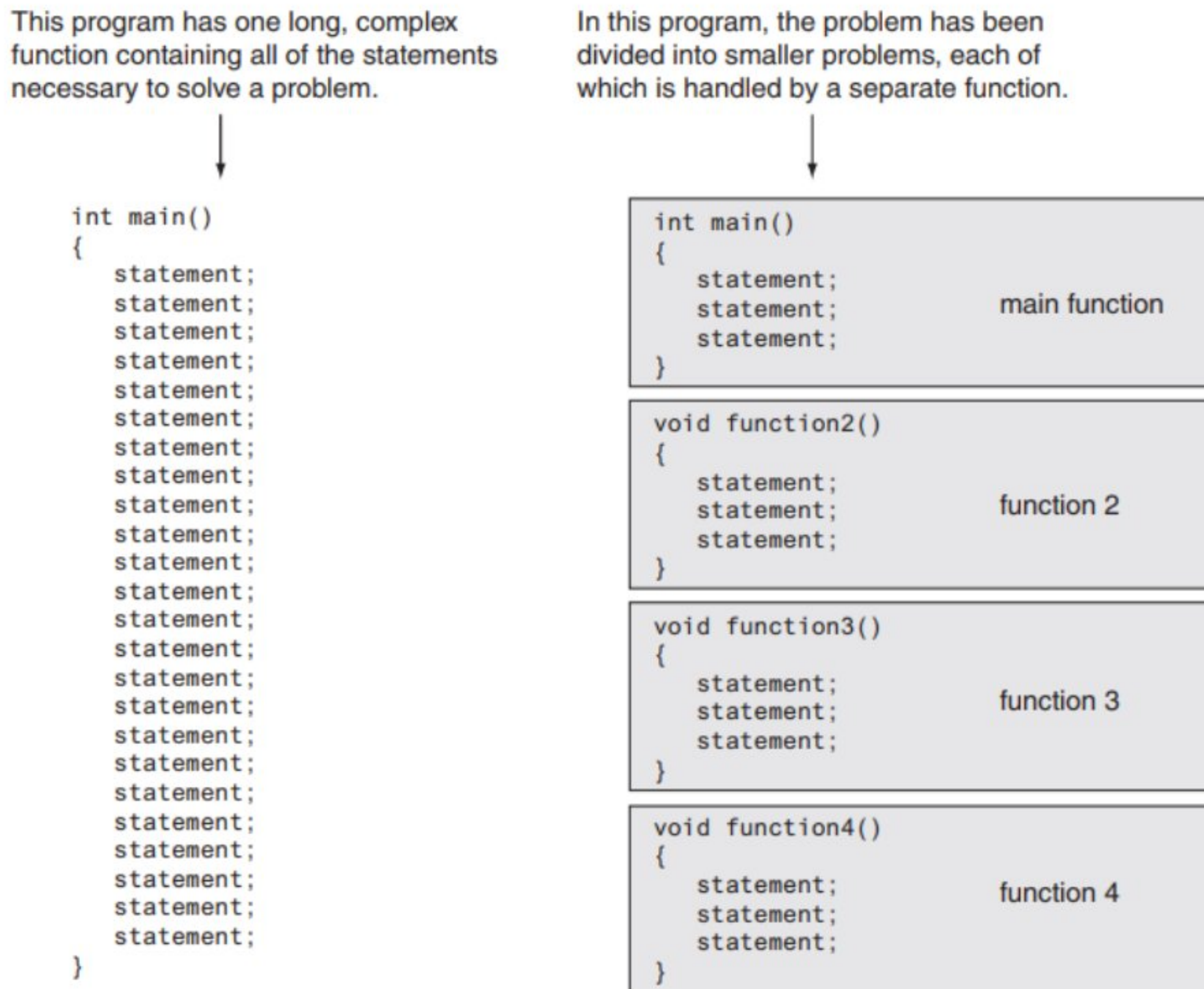
6.1 Focus on Software Engineering: Modular Programming

المفهوم: يمكن تقسيم البرنامج إلى وظائف قابلة للإدارة.

تُستخدم الدوال عادةً لتقسيم المسألة إلى أجزاء صغيرة يسهل التعامل معها. فبدلاً من كتابة دالة واحدة طويلة تحتوي على جميع العبارات اللازمة لحل المسألة، يُمكن كتابة عدة دوال صغيرة، كل منها يحل جزءاً محدداً من المسألة. ثم يُمكن تنفيذ هذه الدوال الصغيرة بالترتيب المطلوب لحل المسألة. يُطلق على هذا النهج أحياناً اسم "فرّق تسد" (*divide and conquer*).

يوضح الشكل 6-1 هذه الفكرة من خلال مقارنة برنامجين: أحدهما يستخدم دالة طويلة ومعقدة تحتوي على جميع العبارات اللازمة لحل مشكلة، والآخر يقسم المشكلة إلى مشاكل أصغر، يتم التعامل مع كل منها بواسطة دالة منفصلة.

Figure 6-1 Modular programming



من أسباب كتابة الدوال أيضاً تبسيطها للبرامج. فإذا نُقِذَت مهمة محددة في عدة مواضع بالبرنامج، يُمكن كتابة دالة مرة واحدة لتنفيذها، ثم تنفيذها عند الحاجة. تُعرف هذه الميزة بإعادة استخدام الكود (Code Reuse)، لأنك تكتب الكود لتنفيذ مهمة مرة واحدة، ثم تُعيد استخدامه في كل مرة تحتاج فيها إلى تنفيذ المهمة.

6.2 Defining and Calling Functions

المفهوم: استدعاء الدالة هو عبارة عن جملة تُنفَّذ. يحتوي تعريف الدالة على الجمل التي تُشكِّلها. عند إنشاء دالة، يجب كتابة تعريفها. تتكون جميع تعريفات الدوال من الأجزاء التالية:

نوع الإرجاع (Return Type): يمكن للدالة إرسال قيمة إلى جزء البرنامج الذي نَفَّذها. نوع الإرجاع هو نوع بيانات القيمة المرسلة من الدالة.

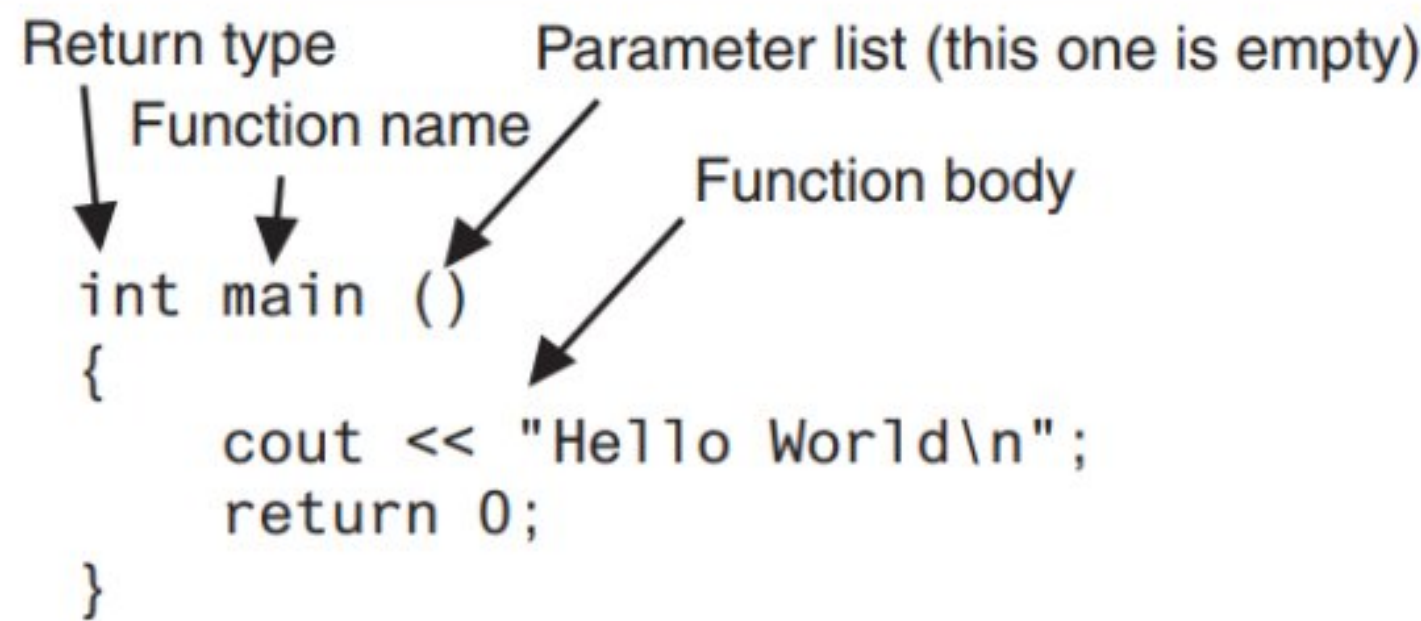
الاسم (Name): يجب إعطاء كل دالة اسمًا وصفيًا. بشكل عام، تنطبق القواعد نفسها التي تنطبق على أسماء المتغيرات على أسماء الدوال أيضًا.

قائمة المعاملات (Parameter List): يمكن للبرنامج إرسال بيانات إلى دالة. قائمة المعاملات هي قائمة من المتغيرات التي تحتوي على القيم التي يتم تمريرها إلى الدالة.

الجسم (Body): جسم الدالة هو مجموعة العبارات التي تُنفَّذ عملية الدالة. وهي محاطة بمجموعة من الأقواس.

يوضح الشكل 6-2 تعريف دالة بسيطة مع تسمية أجزائها المختلفة.

Figure 6-2 Function definition



The line in the definition that reads `int main()` is called the *function header*.

void Functions

ليس من الضروري أن تُرجع جميع الدوال قيمة. بعض الدوال تُنفَّذ جملة واحدة أو أكثر، ثم تُنهيها. تُسمى هذه الدوال دوالاً فارغة. دالة `displayMessage`، التالية، مثال على ذلك.

```
void displayMessage()
{
    cout << "Hello from the function displayMessage.\n";
}
```


اسم الدالة هو `displayMessage`. يُشير هذا الاسم إلى وظيفتها: فهي تعرض رسالة. يجب عليك دائماً تسمية الدوال بأسماء تعكس غرضها. لاحظ أن نوع إرجاع الدالة هو `void`. هذا يعني أن الدالة لا تُرجع قيمة إلى جزء البرنامج الذي ننفّذها. لاحظ أيضاً أن الدالة لا تحتوي على عبارة `return`. إنها ببساطة تعرض رسالة على الشاشة وتخرج.

Calling a Function

تُنفّذ الدالة عند استدعائها. تُستدعى الدالة `main` تلقائياً عند بدء تشغيل البرنامج، ولكن يجب تنفيذ جميع الدوال الأخرى عبر عبارات استدعاء الدالة. عند استدعاء دالة، يتفرّع البرنامج إليها ويُنفّذ عباراتها في نصها. لنلق نظرة على البرنامج 1-6، الذي يحتوي على دالتين: `main` و `display Message`.

Program 6-1

```
1 // This program has two functions: main and displayMessage
2 #include <iostream>
3 using namespace std;
4
5 //*****
6 // Definition of function displayMessage *
7 // This function displays a greeting. *
8 //*****
9
10 void displayMessage()
11 {
12     cout << "Hello from the function displayMessage.\n";
13 }
14
15 //*****
16 // Function main *
17 //*****
18
19 int main()
20 {
21     cout << "Hello from main.\n";
22     displayMessage();
23     cout << "Back in function main again.\n";
24     return 0;
25 }
```

Program Output

```
Hello from main.
Hello from the function displayMessage.
Back in function main again.
```

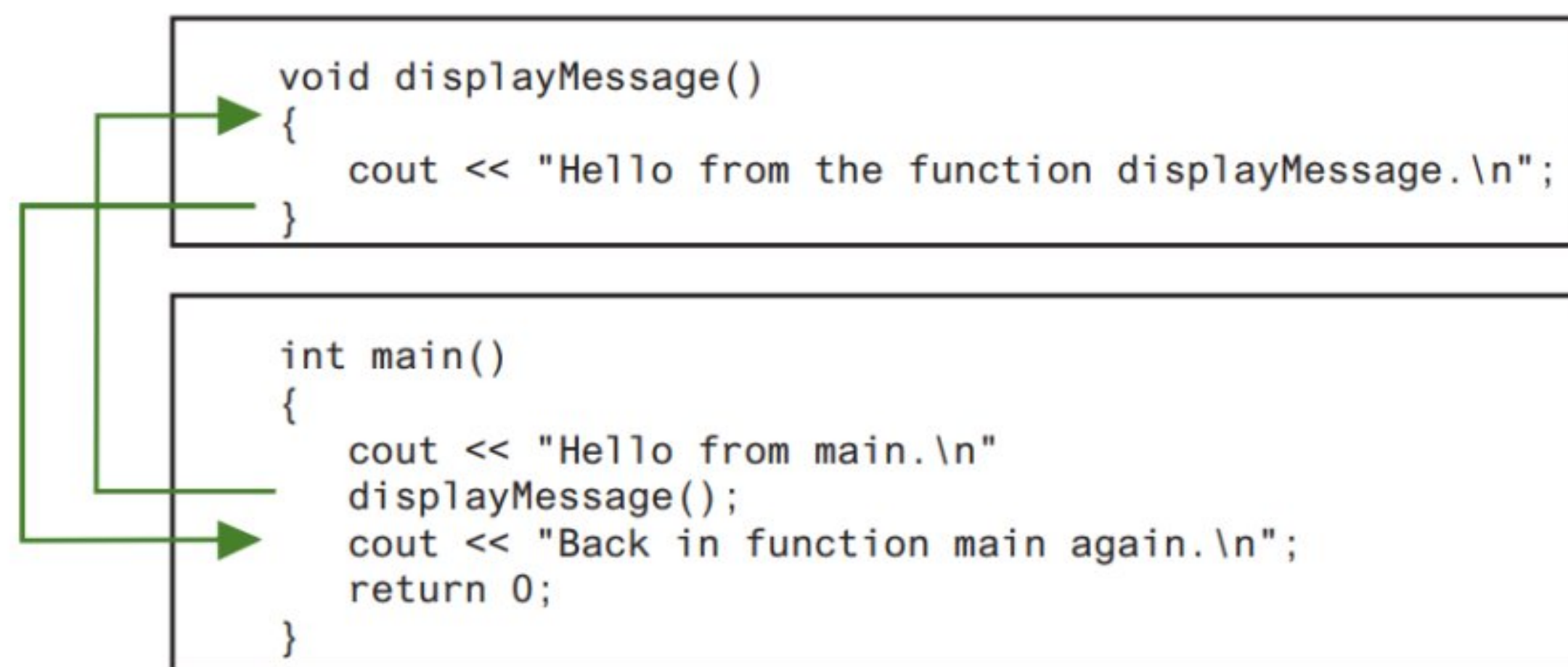

مقارنة بين ترويسة التابع واستدعاء التابع	
Function Header	→ void displayMessage()
Function Call	→ displayMessage();
The function header is: - part of the function definition. - It declares the function's return type, name, and parameter list. - It is not terminated with a semicolon because the definition of the function's body follows it.	The function call is: - a statement that executes the function, so it is terminated with a semicolon like all other C++ statements. - The return type is not listed in the function call. - If the program is not passing data into the function, the parentheses are left empty.

على الرغم من أن البرنامج يبدأ التنفيذ في main، إلا أن الدالة displayMessage تُعرّف أولاً. وذلك لأن المُجمِّع (Compiler) يجب أن يعرف نوع إرجاع الدالة، وعدد المعاملات، ونوع كل معامل قبل استدعائها. إحدى طرق ضمان معرفة المُجمِّع بهذه المعلومات هي وضع تعريف الدالة قبل جميع استدعاءات هذه الدالة. (ستجد لاحقاً طريقة بديلة ومفضلة لتحقيق ذلك).

ملاحظة: يجب عليك دائماً توثيق دوالك بكتابة تعليقات تصف وظيفتها. يجب أن تظهر هذه التعليقات قبل تعريف الدالة مباشرة.

لاحظ كيف يتدفق البرنامج 1-6. يبدأ، بالطبع، في الدالة main. عند استدعاء displayMessage، يتفرّع البرنامج إلى تلك الدالة ويُنفَّذ تعليماتها. بعد انتهاء تنفيذ displayMessage، يتفرّع البرنامج مجدداً إلى الدالة main ويستأنف العمل بالسطر الذي يلي استدعاء الدالة. هذا موضح في الشكل 3-6.

Figure 6-3 A function call



يمكن استخدام عبارات استدعاء الدالة في هياكل التحكم مثل الحلقات، وعبارات الشرط، وعبارات التبديل. يضع البرنامج 2-6 عبارات استدعاء الدالة displayMessage داخل حلقة.

Program 6-2

```
1 // The function displayMessage is repeatedly called from a loop.
2 #include <iostream>
3 using namespace std;
4
5 /*******
6 // Definition of function displayMessage *
7 // This function displays a greeting. *
8 /*******
9
10 void displayMessage()
11 {
12     cout << "Hello from the function displayMessage.\n";
13 }
14
15 /*******
16 // Function main *
17 /*******
18
19 int main()
20 {
21     cout << "Hello from main.\n";
22     for (int count = 0; count < 5; count++)
23         displayMessage(); // Call displayMessage
24     cout << "Back in function main again.\n";
25     return 0;
26 }
```

Program Output

```
Hello from main.
Hello from the function displayMessage.
Hello from the function displayMessage.
Hello from the function displayMessage.
Hello from the function displayMessage.
Hello from the function displayMessage.
Back in function main again.
```

Using Multiple Functions in a Program

من الممكن أن يحتوي البرنامج على العديد من الدوال واستدعاءات الدوال. يحتوي البرنامج 3-6 على ثلاث دوال: الرئيسية، والأولى، والثانية.

Program 6-3

```
1 // This program has three functions: main, first, and second.
2 #include <iostream>
3 using namespace std;
4
```



```

5 //*****
6 // Definition of function first      *
7 // This function displays a message. *
8 //*****
9
10 void first()
11 {
12     cout << "I am now inside the function first.\n";
13 }
14
15 //*****
16 // Definition of function second    *
17 // This function displays a message. *
18 //*****
19
20 void second()
21 {
22     cout << "I am now inside the function second.\n";
23 }
24
25 //*****
26 // Function main                    *
27 //*****
28
29 int main()
30 {
31     cout << "I am starting in function main.\n";
32     first();    // Call function first
33     second();   // Call function second
34     cout << "Back in function main again. \n";
35     return 0;
36 }

```

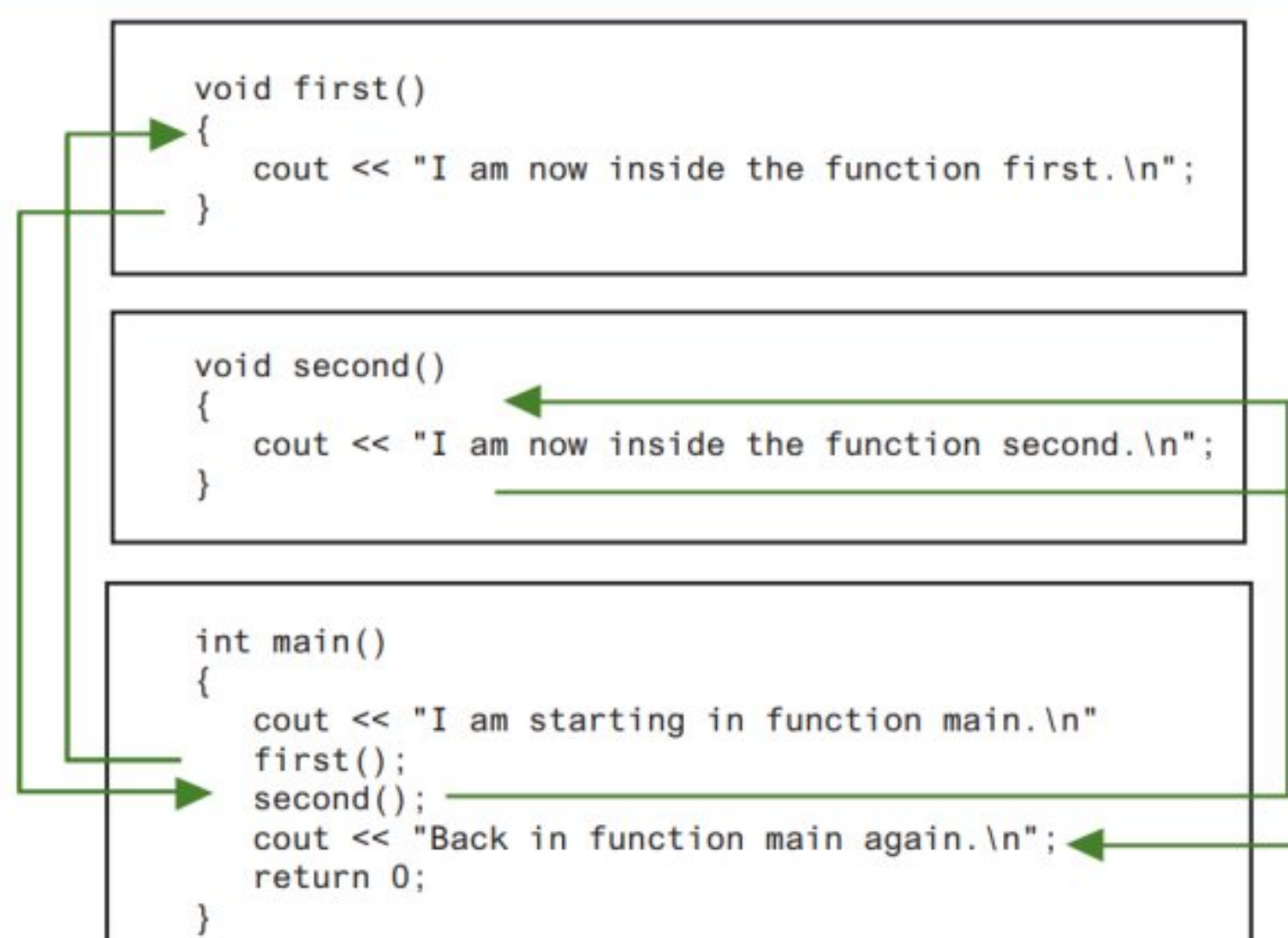
Program Output

```

I am starting in function main.
I am now inside the function first.
I am now inside the function second.
Back in function main again.

```

Figure 6-4 Paths taken by the program



Layered Function Calls

يمكن أيضًا استدعاء الدوال بشكل هرمي أو طبقي. ويوضح البرنامج 4-6 ذلك، إذ يحتوي على ثلاث دوال: رئيسية، وعميقة، وأعمق.

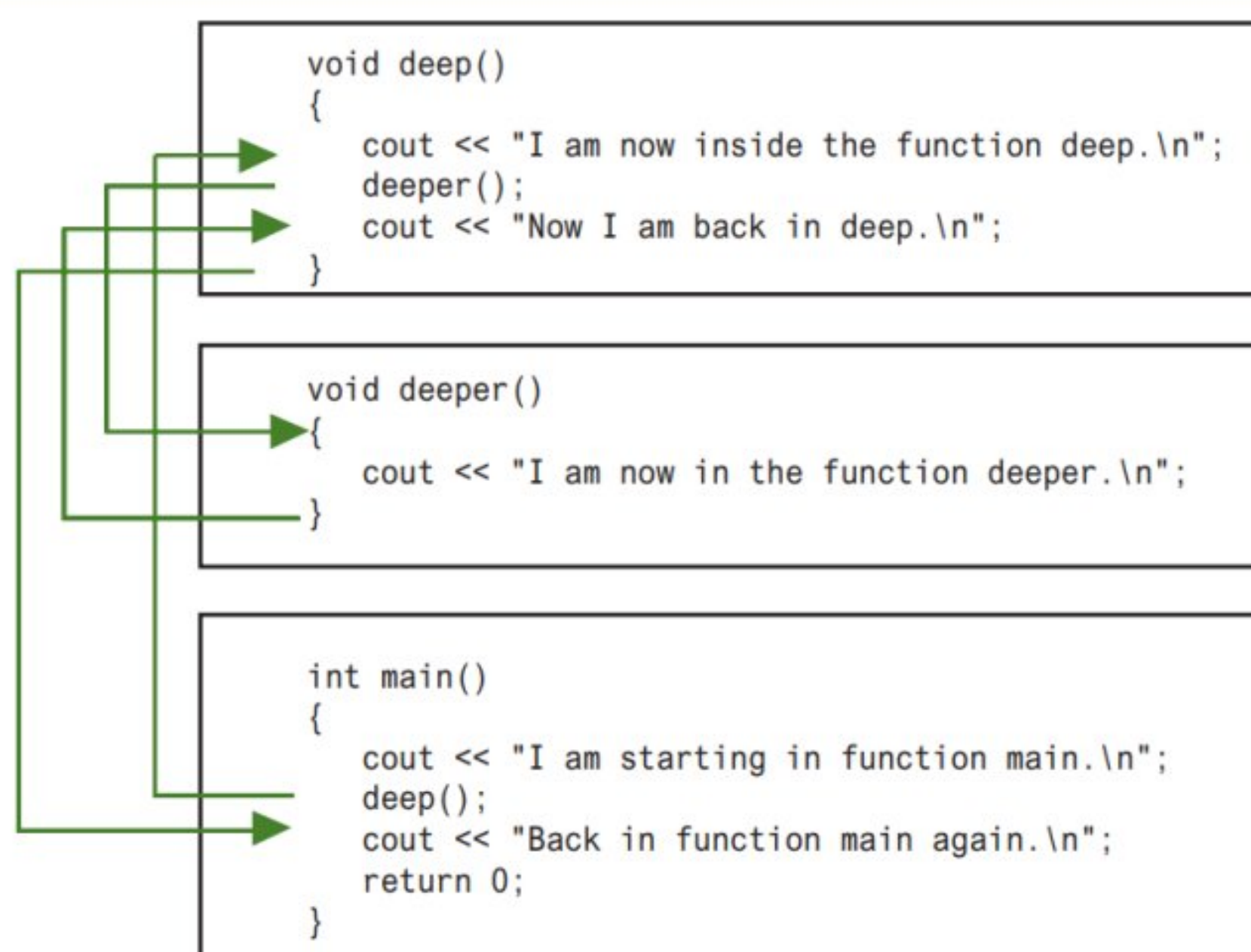
Program 6-4

```
1 // This program has three functions: main, deep, and deeper
2 #include <iostream>
3 using namespace std;
4
5 //*****
6 // Definition of function deeper *
7 // This function displays a message. *
8 //*****
9
10 void deeper()
11 {
12     cout << "I am now inside the function deeper.\n";
13 }
14
15 //*****
16 // Definition of function deep *
17 // This function displays a message. *
18 //*****
19
20 void deep()
21 {
22     cout << "I am now inside the function deep.\n";
23     deeper(); // Call function deeper
24     cout << "Now I am back in deep.\n";
25 }
26
27 //*****
28 // Function main *
29 //*****
30
31 int main()
32 {
33     cout << "I am starting in function main.\n";
34     deep(); // Call function deep
35     cout << "Back in function main again.\n";
36     return 0;
37 }
```

Program Output

```
I am starting in function main.
I am now inside the function deep.
I am now inside the function deeper.
Now I am back in deep.
Back in function main again.
```


Figure 6-5 Layered function calls



6.3 Function Prototypes

المفهوم: يُغني نموذج الدالة الأولي عن وضع تعريف الدالة قبل جميع استدعاءاتها.

قبل أن يواجه المُجمِّع (Compiler) استدعاءً لدالة مُعينة، يجب أن يكون على دراية بنوع إرجاع الدالة، وعدد المُعاملات التي تستخدمها، ونوع كل مُعامل. (ستتعلم كيفية استخدام المُعاملات في القسم التالي).

إحدى طرق ضمان حصول المُجمِّع على هذه المعلومات هي وضع تعريف الدالة قبل جميع استدعاءاتها. كان هذا النهج مُتبعًا في البرامج 1-6، 2-6، 3-6، و4-6. هناك طريقة أخرى وهي إعلان الدالة باستخدام نموذج أولي للدالة. إليك نموذج أولي للدالة `displayMessage` في البرنامج 1-6:

`void displayMessage();`

يبدو النموذج الأولي مشابهًا لرأس الدالة (function header)، باستثناء وجود فاصلة منقوطة في نهايتها.

يُخبر البيان أعلاه المُجمِّع أن الدالة `displayMessage` لها نوع إرجاع `void` (لا تُرجع قيمة) ولا تستخدم أي مُعاملات.

ملاحظة: تُعرف نماذج الدوال (التوابع) أيضًا باسم تصريحات التوابع (Function Declarations).

تحذير! يجب وضع تعريف الدالة أو نموذجها الأولي قبل جميع استدعاءات الدالة. وإلا، فلن يتم ترجمة البرنامج.

عادةً ما تُوضع نماذج التتابع الأولية في أعلى البرنامج، ليتمكن المُجمّع من رؤيتها قبل أي استدعاءات للتتابع. البرنامج 5-6 هو تعديل للبرنامج 3-6. وُضع تعريف الدالتين الأولى والثانية بعد `main`، وُضع نموذج أولي للدالة بعد عبارة `std` الخاصة بمساحة الأسماء `using`.

Program 6-5

```

1 // This program has three functions: main, first, and second.
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototypes
6 void first();
7 void second();
8
9 int main()
10 {
11     cout << "I am starting in function main.\n";
12     first();    // Call function first
13     second();   // Call function second
14     cout << "Back in function main again.\n";
15     return 0;
16 }
17
18 //*****
19 // Definition of function first.      *
20 // This function displays a message. *
21 //*****
22
23 void first()
24 {
25     cout << "I am now inside the function first.\n";
26 }
27
28 //*****
29 // Definition of function second.     *
30 // This function displays a message. *
31 //*****
32
33 void second()
34 {
35     cout << "I am now inside the function second.\n";
36 }

```

Program Output

(The program's output is the same as the output of Program 6-3.)

عند قراءة المُجمّع (Compiler) للبرنامج من 5-6، فإنه يواجه استدعاءات الدالتين الأولى والثانية في السطرين 12 و 13 قبل قراءة تعريفهما. ولكن، بفضل نماذج الدالتين الأوليتين، يعرف المُجمّع نوع الإرجاع ومعلومات معلماتهما (قائمة متغيراتها).
ملاحظة: على الرغم من أن بعض المبرمجين يجعلون `main` هي الوظيفة الأخيرة في البرنامج، إلا أن الكثيرين يفضلون أن تكون الأولى لأنها نقطة بداية البرنامج.

6.4 Sending Data into a Function

المفهوم: عند استدعاء دالة، قد يُرسل البرنامج قيمًا إليها. تُسمى القيم المُرسلة إلى دالة "مُعاملات". أنت مُلمٌ بكيفية استخدام المُعاملات في استدعاء دالة. في العبارة التالية، تُستدعى الدالة pow، ويُمرَّر إليها مُعاملان، 2.0 و4.0:

```
result = pow(4.0, 2.0);
```

باستخدام المُعاملات، يُمكنك تصميم دوال خاصة بك تقبل البيانات بهذه الطريقة. المُعامل هو مُتغير خاص يحمل قيمة تُمرَّر إلى دالة. إليك تعريف الدالة التي تستخدم مُعاملًا:

```
void displayValue (int num)
{
    cout << "The value is " << num << endl;
}
```

لاحظ تعريف متغير العدد الصحيح داخل الأقواس (int num). المتغير num هو مُعامل. يُمكن هذا الدالة displayValue من قبول قيمة عدد صحيح كُعامل. البرنامج 6-6 هو برنامج كامل يستخدم هذه الدالة.

ملاحظة: في هذا النص، تُسمى القيم التي تُمرَّر إلى دالة ما "مُعاملات"، وتُسمى المتغيرات التي تستقبل هذه القيم "مُعاملات". هناك عدة أشكال لهذه المصطلحات قيد الاستخدام. يُطلق البعض على هذه المُعاملات "مُعاملات فعلية" ويُطلق عليها "مُعاملات شكلية". بينما يستخدم آخرون مصطلحي "مُعامل فعلي" و"مُعامل شكلي". بغض النظر عن مجموعة المصطلحات التي تستخدمها، من المهم أن تكون متسقًا.

Program 6-6

```
1 // This program demonstrates a function with a parameter.
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void displayValue(int);
7
8 int main()
9 {
10     cout << "I am passing 5 to displayValue.\n";
11     displayValue(5); // Call displayValue with argument 5
12     cout << "Now I am back in main.\n";
13     return 0;
14 }
15
```


Program 6-6*(continued)*

```

16  // *****
17  // Definition of function displayValue.
18  // It uses an integer parameter whose value is displayed.*
19  // *****
20
21  void displayValue(int num)
22  {
23      cout << "The value is " << num << endl;
24  }

```

Program Output

I am passing 5 to displayValue.
The value is 5
Now I am back in main.

Figure 6-6 An argument passed to a function

displayValue(5);

void displayValue(int num)
{
cout << "The value is " << num << endl;
}

أي وسيطة مُدرجة بين قوسي استدعاء دالة تُنسخ إلى متغير معلماتها. في جوهرها، تُهيأ متغيرات المعلمات بقيمة معلماتها المقابلة. يُظهر البرنامج 7-6 استدعاء الدالة displayValue عدة مرات، مع تمرير معلمة مختلفة في كل مرة.

Program 6-7

```

1  // This program demonstrates a function with a parameter.
2  #include <iostream>
3  using namespace std;
4
5  // Function Prototype
6  void displayValue(int);
7

```



```

8  int main()
9  {
10     cout << "I am passing several values to displayValue.\n";
11     displayValue(5); // Call displayValue with argument 5
12     displayValue(10); // Call displayValue with argument 10
13     displayValue(2); // Call displayValue with argument 2
14     displayValue(16); // Call displayValue with argument 16
15     cout << "Now I am back in main.\n";
16     return 0;
17 }
18
19 //*****
20 // Definition of function displayValue. *
21 // It uses an integer parameter whose value is displayed.*
22 //*****
23
24 void displayValue(int num)
25 {
26     cout << "The value is " << num << endl;
27 }

```

Program Output

```

I am passing several values to displayValue.
The value is 5
The value is 10
The value is 2
The value is 16
Now I am back in main.

```

تحذير! عند تمرير متغير كمعامل، اكتب اسم المتغير ببساطة داخل أقواس استدعاء الدالة. لا تكتب نوع بيانات متغير العامل في استدعاء الدالة. على سبيل المثال، سيؤدي استدعاء الدالة التالي إلى حدوث خطأ:

displayValue(int x); // Error!

يجب أن يظهر استدعاء الدالة بالشكل التالي:

displayValue(x); // Correct

في كل مرة تُستدعى فيها الدالة في البرنامج 6-7، تأخذ الدالة num قيمة مختلفة. يمكن استخدام أي تعبير يُمكن عادةً تعيين قيمته إلى num كمعامل. على سبيل المثال، سيؤدي استدعاء الدالة التالي إلى تمرير القيمة 8 إلى num:

displayValue(3 + 5);

إذا مررت وسيطةً يختلف نوعها عن نوع المُعامل، فسيتم ترقية الوسيطة (promoted) أو تخفيض رتبته (demoted) تلقائيًا. على سبيل المثال، سيتم اقتطاع الوسيطة (truncated) في استدعاء الدالة التالي، مما يؤدي إلى تمرير القيمة 4 إلى num:

displayValue(4.7);

Passing Multiple Arguments

في كثير من الأحيان، من المفيد تمرير عدة وسيطات إلى دالة. يوضح البرنامج 6-8 تعريف دالة بثلاثة معاملات.

Program 6-8

```
1 // This program demonstrates a function with three parameters.
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void showSum(int, int, int);
7
8 int main()
9 {
10     int value1, value2, value3;
11
12     // Get three integers.
13     cout << "Enter three integers and I will display ";
14     cout << "their sum: ";
15     cin >> value1 >> value2 >> value3;
16
17     // Call showSum passing three arguments.
18     showSum(value1, value2, value3);
19     return 0;
20 }
21
22 //*****
23 // Definition of function showSum. *
24 // It uses three integer parameters. Their sum is displayed. *
25 //*****
26
27 void showSum(int num1, int num2, int num3)
28 {
29     cout << (num1 + num2 + num3) << endl;
30 }
```

Program Output with Example Input Shown in Bold

Enter three integers and I will display their sum: **4 8 7**
19

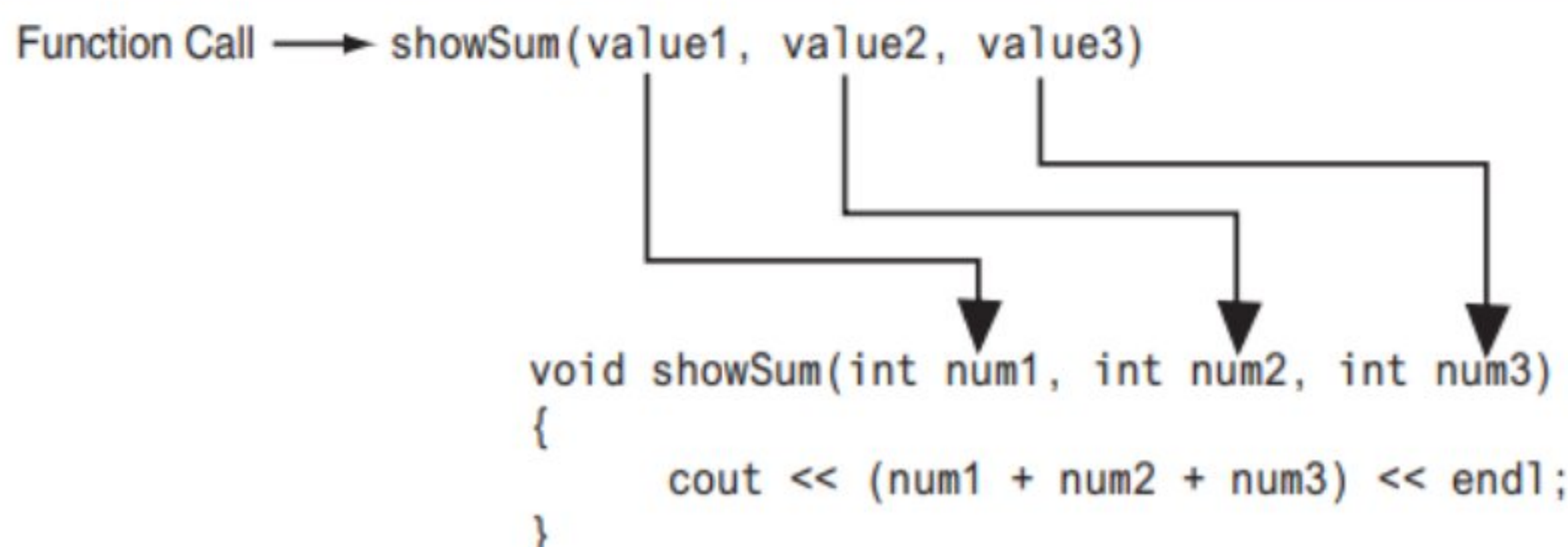
تحذير! يجب أن يُدرج نوع بيانات كل متغير معلمة في قائمة المَعْلَمَات قبل اسمه. على سبيل المثال، سيحدث خطأ في المترجم إذا عُرِفَت قائمة مَعْلَمَات دالة showSum كما هو موضح في العنوان التالي:

```
void showSum(int num1, num2, num3) // Error!
```

يجب إدراج نوع بيانات جميع متغيرات المَعْلَمَات الثلاثة، كما هو موضح هنا:

```
void showSum(int num1, int num2, int num3) // Correct
```

Figure 6-7 Multiple arguments



ملاحظة: يجب أن يسرد النموذج الأولي للتابع (function prototype) نوع البيانات لكل معلمة (parameter).

ملاحظة: كما هو الحال مع جميع المتغيرات، للمَعْلَمَات (parameters) نطاق. يقتصر نطاق المعلمة على جسم (body) الدالة التي تستخدمها.

6.5 Passing Data by Value

المفهوم: عند تمرير وسيطة (argument) إلى مُعامل (parameter)، يتم تمرير نسخة من قيمة الوسيطة فقط. لا تؤثر تغييرات المُعامل على الوسيطة الأصلية.

كما رأيت في هذه المحاضرة، المعاملات هي متغيرات ذات أغراض خاصة تُعرّف داخل أقواس تعريف الدالة. وهي منفصلة ومتميزة عن الوسائط المُدرجة داخل أقواس استدعاء الدالة. القيم المُخزّنة في متغيرات المعاملات هي نسخ من الوسائط. عادةً، عند تغيير قيمة مُعامل داخل دالة، لا يؤثر ذلك على الوسيط الأصلي. يوضح البرنامج 6-9 هذا المفهوم.

Program 6-9

```
1 // This program demonstrates that changes to a function parameter
2 // have no effect on the original argument.
3 #include <iostream>
4 using namespace std;
5
6 // Function Prototype
7 void changeMe(int);
8
9 int main()
10 {
11     int number = 12;
12
13     // Display the value in number.
14     cout << "number is " << number << endl;
15
16     // Call changeMe, passing the value in number
17     // as an argument.
18     changeMe(number);
19
20     // Display the value in number again.
21     cout << "Now back in main again, the value of ";
22     cout << "number is " << number << endl;
23     return 0;
24 }
25
26 //*****
27 // Definition of function changeMe. *
28 // This function changes the value of the parameter myValue. *
29 //*****
30
31 void changeMe(int myValue)
32 {
33     // Change the value of myValue to 0.
34     myValue = 0;
35
36     // Display the value in myValue.
37     cout << "Now the value is " << myValue << endl;
38 }
```

Program Output

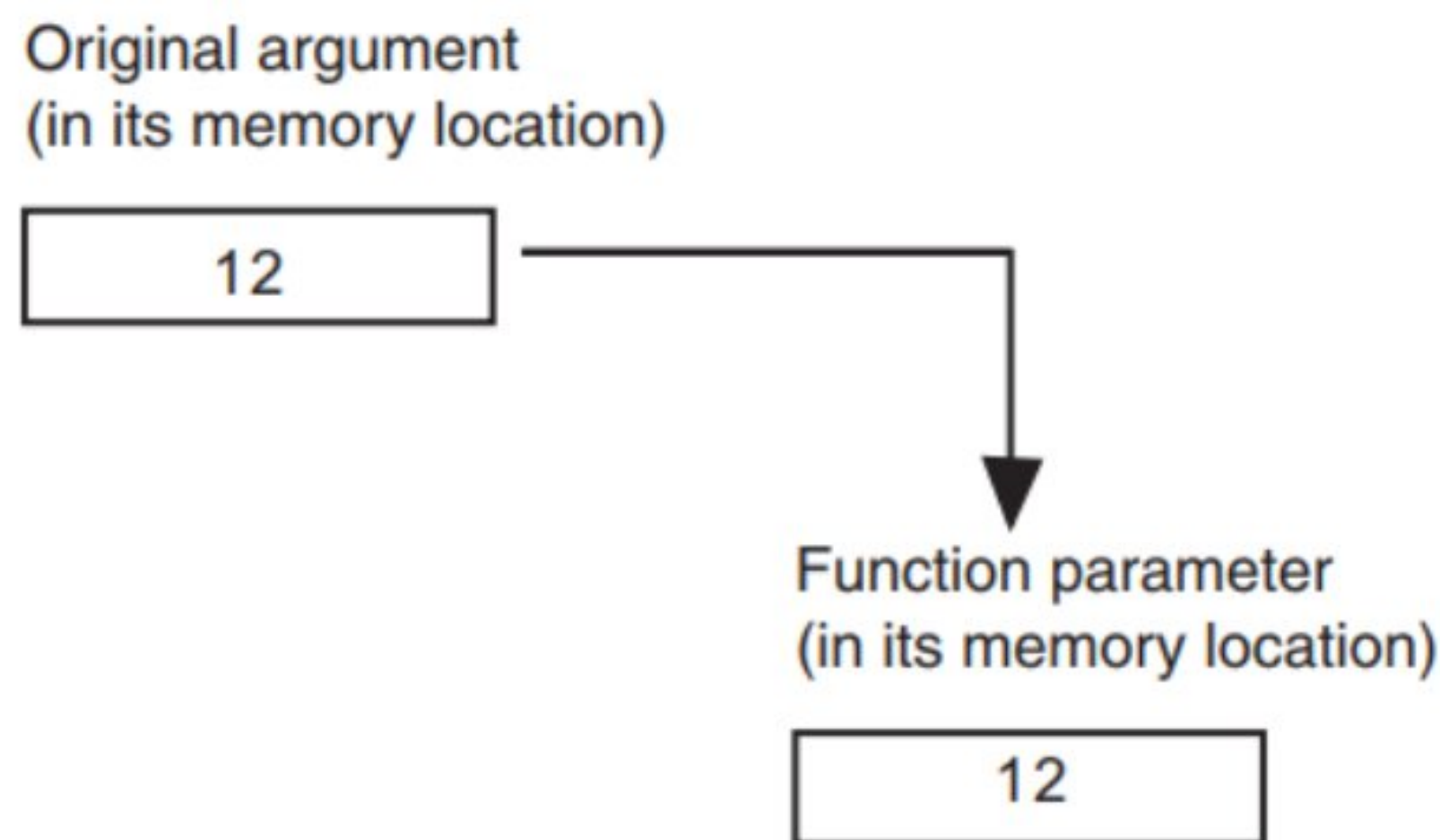
```
number is 12
Now the value is 0
Now back in main again, the value of number is 12
```

على الرغم من تغيير متغير المعلمة myValue في دالة changeMe، إلا أن رقم الوسيطة لا يتغير. يحتوي متغير myValue على نسخة فقط من متغير الرقم.

لا تستطيع دالة changeMe الوصول إلى الوسيطة الأصلية. عند تمرير نسخة فقط من وسيطة إلى دالة، يُقال إنها مُمررة بالقيمة (passed by value). وذلك لأن الدالة تستقبل نسخة من قيمة الوسيطة ولا تستطيع الوصول إلى الوسيطة الأصلية.

يوضح الشكل 6-8 أن موقع تخزين متغير المعلمة في الذاكرة منفصل عن موقع الوسيطة الأصلية.

Figure 6-8 The argument and the parameter are separate



6.6 Focus on Software Engineering: Using Functions in a Menu-Driven Program

المفهوم: تُعد الدوال مثالية للاستخدام في البرامج التي تستخدم قائمة. فعندما يختار المستخدم عنصرًا من القائمة، يستطيع البرنامج استدعاء الدالة المناسبة.

يُظهر البرنامج 6-10 إعادة تصميم البرنامج ليصبح برنامجًا معياريًا. يُقسّم البرنامج المعياري إلى وظائف تُنفَّذ مهامًا محددة.

Program 6-10

```

1 // This is a menu-driven program that makes a function call
2 // for each selection the user makes.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 // Function prototypes
8 void showMenu();
9 void showFees(double, int);
10
11 int main()
12 {
13     int choice;           // To hold a menu choice
14     int months;          // To hold a number of months
15
  
```



```

16 // Constants for the menu choices
17 const int ADULT_CHOICE = 1,
18         CHILD_CHOICE = 2,
19         SENIOR_CHOICE = 3,
20         QUIT_CHOICE = 4;
21
22 // Constants for membership rates
23 const double ADULT = 40.0,
24             CHILD = 20.0;
25             SENIOR = 30.0;
26
27 // Set up numeric output formatting.
28 cout << fixed << showpoint << setprecision(2);
29
30 do
31 {
32     // Display the menu and get the user's choice.
33     showMenu();
34     cin >> choice;
35
36     // Validate the menu selection.
37     while (choice < ADULT_CHOICE || choice > QUIT_CHOICE)
38     {
39         cout << "Please enter a valid menu choice: ";
40         cin >> choice;
41     }
42
43     // If the user does not want to quit, proceed.
44     if (choice != QUIT_CHOICE)
45     {
46         // Get the number of months.
47         cout << "For how many months? ";
48         cin >> months;
49
50         // Display the membership fees.
51         switch (choice)
52         {
53             case ADULT_CHOICE:
54                 showFees(ADULT, months);
55                 break;
56             case CHILD_CHOICE:
57                 showFees(CHILD, months);
58                 break;
59             case SENIOR_CHOICE:
60                 showFees(SENIOR, months);
61         }
62     }
63     while (choice != QUIT_CHOICE);
64     return 0;
65 }
66
67 //*****
68 // Definition of function showMenu which displays the menu.      *
69 //*****
70
71 void showMenu()
72 {
73     cout << "\n\t\tHealth Club Membership Menu\n\n"
74         << "1. Standard Adult Membership\n"
75         << "2. Child Membership\n"
76         << "3. Senior Citizen Membership\n"
77         << "4. Quit the Program\n\n"
78         << "Enter your choice: ";
79 }
80

```



```

81 //*****
82 // Definition of function showFees. The memberRate parameter holds *
83 // the monthly membership rate and the months parameter holds the *
84 // number of months. The function displays the total charges.      *
85 //*****
86
87 void showFees(double memberRate, int months)
88 {
89     cout << "The total charges are $"
90         << (memberRate * months) << endl;
91 }

```

(program output continues)

Program 6-10 (continued)

Program Output with Example Input Shown in Bold

```

Health Club Membership Menu

1. Standard Adult Membership
2. Child Membership
3. Senior Citizen Membership
4. Quit the Program

Enter your choice: 1 
For how many months? 12 
The total charges are $480.00

```

```

Health Club Membership Menu

1. Standard Adult Membership
2. Child Membership
3. Senior Citizen Membership
4. Quit the Program

Enter your choice: 4 

```

6.7 The return Statement

المفهوم: تؤدي عبارة الإرجاع إلى إنهاء الدالة فوراً.

عند انتهاء تنفيذ آخر عبارة في دالة فارغة، تنتهي الدالة ويعود البرنامج إلى العبارة التي تلت استدعائها. مع ذلك، من الممكن إجبار الدالة على العودة قبل تنفيذ آخر عبارة. عند مواجهة عبارة الإرجاع، تنتهي الدالة فوراً ويعود التحكم بالبرنامج إلى العبارة التي استدعت الدالة. هذا موضح في البرنامج 6-11.

يوضح قسمة الدالة حاصل قسمة `arg1` على `arg2`. إذا تم ضبط `arg2` على صفر، تعود الدالة.

Program 6-11

```
1 // This program uses a function to perform division. If division
2 // by zero is detected, the function returns.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototype.
7 void divide(double, double);
8
9 int main()
10 {
11     double num1, num2;
12
13     cout << "Enter two numbers and I will divide the first\n";
14     cout << "number by the second number: ";
15     cin >> num1 >> num2;
16     divide(num1, num2);
17     return 0;
18 }
19
20 //*****
21 // Definition of function divide. *
22 // Uses two parameters: arg1 and arg2. The function divides arg1 *
23 // by arg2 and shows the result. If arg2 is zero, however, the *
24 // function returns. *
25 //*****
26
27 void divide(double arg1, double arg2)
28 {
29     if (arg2 == 0.0)
30     {
31         cout << "Sorry, I cannot divide by zero.\n";
32         return;
33     }
34     cout << "The quotient is " << (arg1 / arg2) << endl;
35 }
```

Program Output with Example Input Shown in Bold

```
Enter two numbers and I will divide the first
number by the second number: 12 0 Enter
Sorry, I cannot divide by zero.
```

في مثال تشغيل البرنامج، قام المستخدم بإدخال 12 و 0 كمدخل. في السطر 16، تم استدعاء دالة القسمة، وتم تمرير 12 إلى المعلمة arg1 و 0 إلى المعلمة arg2. المعلمة داخل دالة القسمة، يتم تنفيذ عبارة if في السطر 29. لأن arg2 يساوي 0.0، ويتم تنفيذ الكود الموجود في السطرين 31 و 32. عندما يكون بيان الإرجاع في السطر 32 يتم تنفيذه، وينتهي تابع القسمة على الفور. وهذا يعني عبارة cout في السطر 34 لا ينفذ. يستأنف البرنامج عند السطر 17 في التابع الرئيسي.

6.8 Returning a Value from a Function

المفهوم: قد تُرسل الدالة قيمةً إلى جزء البرنامج الذي استدعاها.

لقد رأيت أنه يمكن تمرير البيانات إلى الدالة عبر متغيرات المعاملات. كما يمكن إرجاع البيانات من الدالة، إلى العبارة التي استدعتها. تُعرف الدوال التي تُرجع قيمةً باسم دوال إرجاع القيمة (value-returning functions).

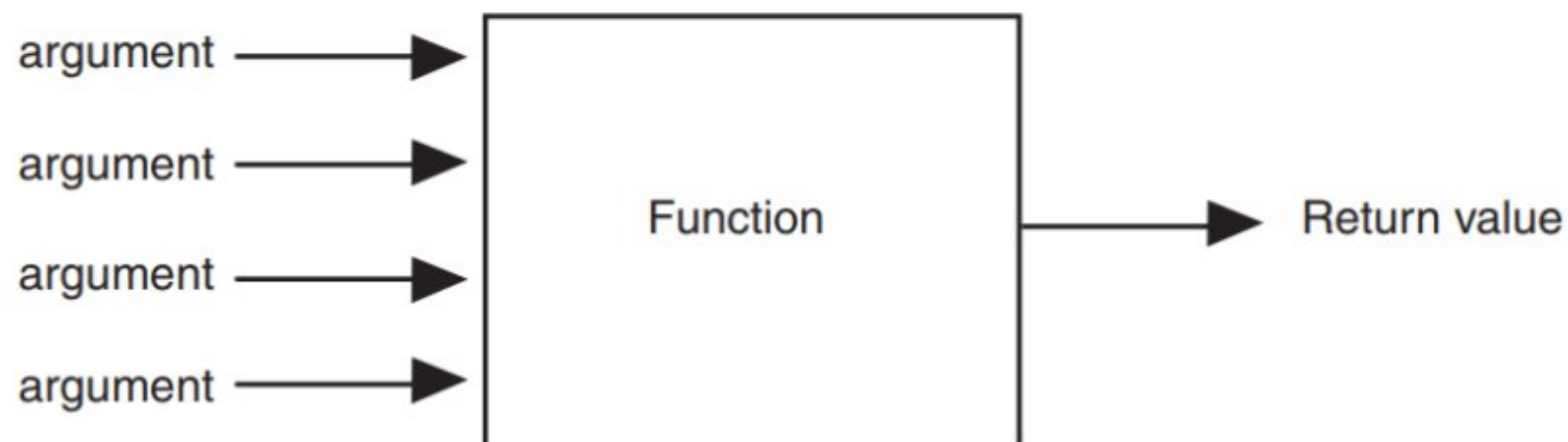
دالة pow، التي سبق أن رأيتها، هي مثال على دالة إرجاع القيمة. إليك مثال:

```
double x;  
x = pow(4.0, 2.0);
```

يستدعي السطر الثاني من هذا الكود دالة pow، ويمرر القيمتين 4.0 و 2.0 كوسيطين. تحسب الدالة قيمة العدد 4.0 مرفوعةً للأس 2.0، وتُرجعها. تُخصص القيمة، وهي 16.0، للمتغير x بواسطة عامل الإسناد =.

على الرغم من إمكانية تمرير عدة وسيطات إلى دالة، إلا أنه لا يمكن إرجاع سوى قيمة واحدة منها. تخيل أن الدالة تحتوي على قنوات اتصال متعددة لاستقبال البيانات (المعاملات)، ولكن قناة واحدة فقط لإرسال البيانات (قيمة الإرجاع). يوضح الشكل 6-9 ذلك.

Figure 6-9 A function returns one value



ملاحظة: من الممكن إرجاع قيم متعددة من دالة، ولكن يجب "تعبئتها" بطريقة يتم التعامل معها كقيمة واحدة وهذا ما سنراه لاحقاً في محاضرة البيانات المهيكلة (structured data).

Defining a Value-Returning Function

عند كتابة دالة إرجاع قيمة، يجب تحديد نوع القيمة التي ستعيدها. وذلك لأنه يجب تحديد نوع بيانات قيمة الإرجاع في رأس الدالة (function header) وفي نموذج الدالة. تذكر أن دالة void، التي لا تُرجع قيمة، تستخدم الكلمة المفتاحية void كنوع إرجاع لها في رأس الدالة.

ستستخدم دالة إرجاع القيمة نوع بيانات صحيح `int`، أو `double`، أو `bool`، أو أي نوع بيانات صالح آخر في رأسها. إليك مثال على دالة تُرجع قيمة صحيحة:

```
int sum (int num1, int num2)
{
    int result;
    result = num1 + num2;
    return result;
}
```

اسم هذه الدالة هو `sum`. لاحظ أن نوع الإرجاع في رأس الدالة هو `int`، كما هو موضح في الشكل 6-10.

Figure 6-10 Function return type

Return type



`int sum(int num1, int num2)`

يُعرّف هذا الكود دالة تُسمى `sum`، تقبل مُعاملين من نوع `int`. تُمرّر المُعاملات إلى مُتغيري المُعاملات `num1` و `num2`. داخل الدالة، يُعرّف مُتغير النتيجة. تُسمى المُتغيرات المُعرّفة داخل الدالة مُتغيرات محلية. بعد تعريف المُتغير، يُضاف مُتغيرا المُعاملات `num1` و `num2`، ويُخصّص مُعامل المجموع الخاص بهما إلى مُتغير النتيجة. العبارة الأخيرة في الدالة هي:

return result;

تؤدي هذه العبارة إلى إنهاء الدالة، وتُعيد قيمة مُتغير النتيجة إلى العبارة التي استدعت الدالة. يجب أن تحتوي دالة إرجاع القيمة على عبارة إرجاع مكتوبة بالتنسيق العام التالي:

`return expression;`

ومع ذلك، كان بإمكاننا حذف مُتغير النتيجة وإرجاع التعبير `num1 + num2`، كما هو موضح في الكود التالي:

```
int sum (int num1, int num2)
{
    return num1 + num2;
}
```

عند كتابة النموذج الأولي لدالة إرجاع القيمة، اتبع نفس القواعد التي تناولناها سابقًا. إليك النموذج الأولي لدالة الجمع:

```
int sum (int, int);
```


Calling a Value-Returning Function

يُظهر البرنامج 12-6 مثالاً لكيفية استدعاء دالة المجموع.

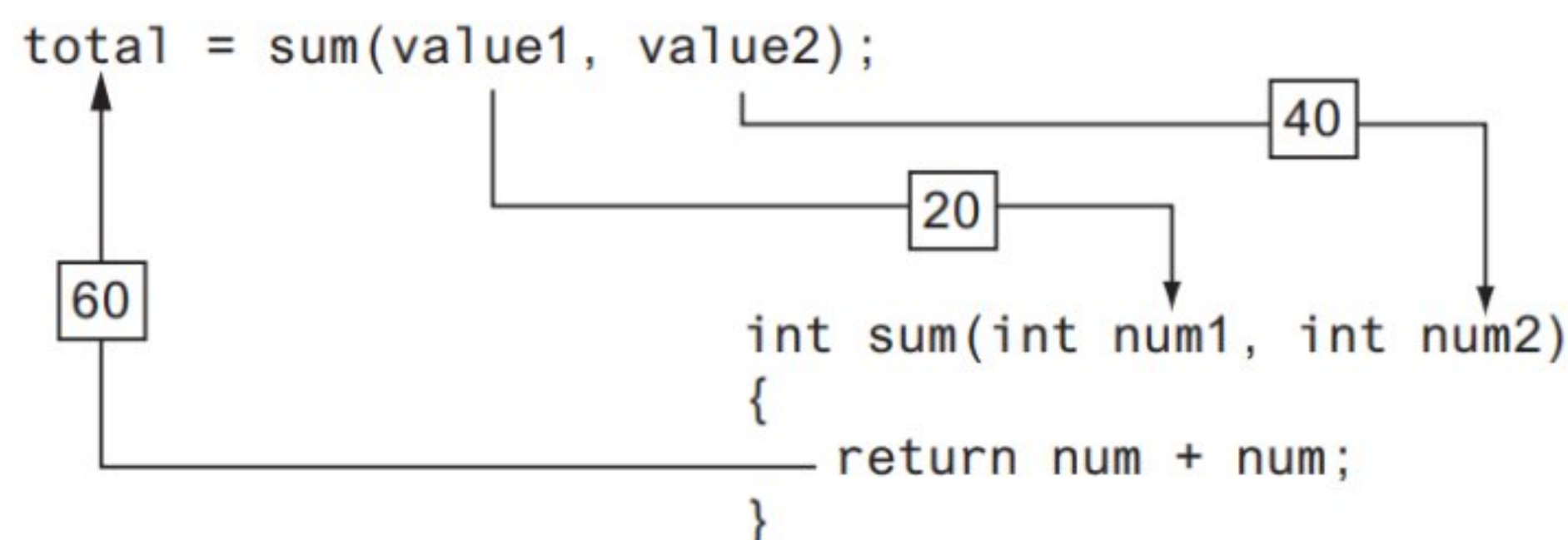
Program 6-12

```
1 // This program uses a function that returns a value.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype
6 int sum(int, int);
7
8 int main()
9 {
10     int value1 = 20,    // The first value
11        value2 = 40,    // The second value
12        total;          // To hold the total
13
14     // Call the sum function, passing the contents of
15     // value1 and value2 as arguments. Assign the return
16     // value to the total variable.
17     total = sum(value1, value2);
18
19     // Display the sum of the values.
20     cout << "The sum of " << value1 << " and "
21          << value2 << " is " << total << endl;
22     return 0;
23 }
24
25 //*****
26 // Definition of function sum. This function returns *
27 // the sum of its two parameters.                      *
28 //*****
29
30 int sum(int num1, int num2)
31 {
32     return num1 + num2;
33 }
```

Program Output

The sum of 20 and 40 is 60

Figure 6-11 A function's arguments and return value



لنلق نظرة على مثال آخر. يحتوي البرنامج 6-13، الذي يحسب مساحة الدائرة، على دالتين بالإضافة إلى الدالة الرئيسية. تُسمى إحداها square، وتُرجع مربع أي عدد يُمرر إليها كمعامل. تُستدعى دالة square في جملة رياضية. يحتوي البرنامج أيضًا على دالة getRadius، والتي تُطالب المستخدم بإدخال نصف قطر الدائرة. تُرجع الدالة القيمة التي يُدخلها المستخدم.

Program 6-13

```
1 // This program demonstrates two value-returning functions.
2 // The square function is called in a mathematical statement.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
```

Program 6-13 (continued)

```
6
7 //Function prototypes
8 double getRadius();
9 double square(double);
10
11 int main()
12 {
13     const double PI = 3.14159; // Constant for pi
14     double radius;             // To hold the circle's radius
15     double area;               // To hold the circle's area
16
17     // Set the numeric output formatting.
18     cout << fixed << showpoint << setprecision(2);
19
20     // Get the radius of the circle.
21     cout << "This program calculates the area of ";
22     cout << "a circle.\n";
23     radius = getRadius();
24
25     // Calculate the area of the circle.
26     area = PI * square(radius);
27
28     // Display the area.
29     cout << "The area is " << area << endl;
30     return 0;
31 }
32
33 //*****
34 // Definition of function getRadius.
35 // This function asks the user to enter the radius of
36 // the circle and then returns that number as a double.*
37 //*****
38
```



```

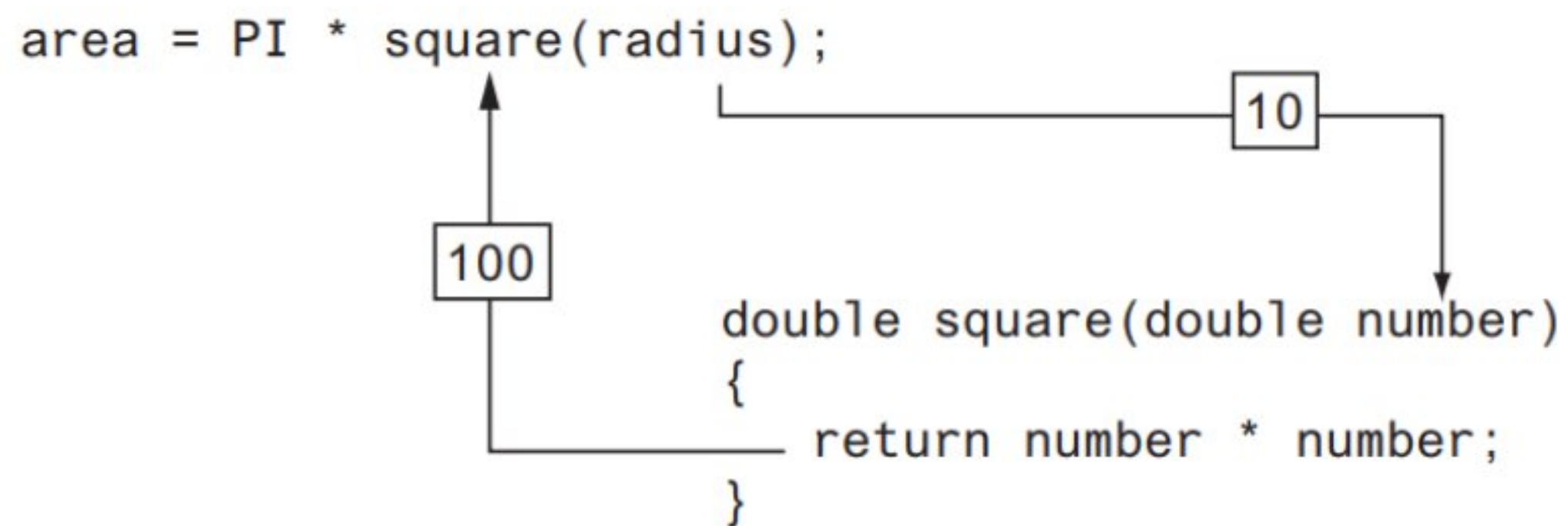
39 double getRadius()
40 {
41     double rad;
42
43     cout << "Enter the radius of the circle: ";
44     cin >> rad;
45     return rad;
46 }
47
48 //*****
49 // Definition of function square.
50 // This function accepts a double argument and returns
51 // the square of the argument as a double.
52 //*****
53
54 double square(double number)
55 {
56     return number * number;
57 }

```

Program Output with Example Input Shown in Bold

This program calculates the area of a circle.
Enter the radius of the circle: **10**
The area is 314.16

Figure 6-12 The square function



6.9 Returning a Boolean Value

المفهوم: قد تُرجع الدوال قيمةً صحيحة أو خاطئة.

في كثير من الأحيان، هناك حاجة إلى دالة تختبر وسيطة وتُرجع قيمةً صحيحة أو خاطئة، مما يُشير إلى وجود شرط من عدمه. تُرجع هذه الدالة قيمة منطقية. على سبيل المثال، تقبل الدالة التالية وسيطة

من نوع int وُترجع القيمة "صحيح" إذا كانت الوسيطة ضمن نطاق من 1 إلى 100، أو "خطأ" بخلاف ذلك:

```
bool isValid (int number)
{
    bool status;
    if (number >= 1 && number <= 100)
        status = true;
    else
        status = false;

    return status;
}
```

يُظهر الكود التالي عبارة if/else التي تستخدم استدعاءً للوظيفة:

```
int value = 20;
if (isValid(value))
    cout << "The value is within range.\n";
else
    cout << "The value is out of range.\n";
```

عند تنفيذ هذا الكود، ستظهر رسالة "The value is within range."

يعرض البرنامج 6-15 مثالاً آخر. يحتوي هذا البرنامج على دالة تُسمى isEven، والتي تُرجع القيمة true إذا كانت مُعاملاتها عدداً زوجياً. وإلا، تُرجع الدالة القيمة false.

Program 6-15

```
1 // This program uses a function that returns true or false.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype
6 bool isEven(int);
7
8 int main()
9 {
10     int val;
11
12     // Get a number from the user.
13     cout << "Enter an integer and I will tell you ";
14     cout << "if it is even or odd: ";
15     cin >> val;
16
17     // Indicate whether it is even or odd.
18     if (isEven(val))
19         cout << val << " is even.\n";
20     else
21         cout << val << " is odd.\n";
22     return 0;
23 }
24
```



```

25 //*****
26 // Definition of function isEven. This function accepts an *
27 // integer argument and tests it to be even or odd. The function *
28 // returns true if the argument is even or false if the argument *
29 // is odd. The return value is a bool. *
30 //*****
31
32 bool isEven(int number)
33 {
34     bool status;
35
36     if (number % 2 == 0)
37         status = true; // The number is even if there is no remainder.
38     else

```

(program continues)

Program 6-15 (continued)

```

39         status = false; // Otherwise, the number is odd.
40     return status;
41 }

```

Program Output with Example Input Shown in Bold

Enter an integer and I will tell you if it is even or odd: **5**
5 is odd.

6.10 Local and Global Variables

المفهوم: يُعرّف المتغير المحلي داخل دالة، ولا يمكن الوصول إليه خارجها. أما المتغير الشامل، فيُعرّف خارج جميع الدوال، ويمكن الوصول إليه من جميع الدوال في نطاقه.

Local Variables

المتغيرات المُعرّفة داخل دالة ما تكون محلية لتلك الدالة. وهي مخفية عن عبارات الدوال الأخرى، والتي عادةً لا تستطيع الوصول إليها. يوضح البرنامج 6-16 أنه نظرًا لإخفاء المتغيرات المُعرّفة في دالة ما، فقد تحتوي الدوال الأخرى على متغيرات منفصلة ومتميزة تحمل الاسم نفسه.

Program 6-16

```

1 // This program shows that variables defined in a function
2 // are hidden from other functions.
3 #include <iostream>
4 using namespace std;

5
6 void anotherFunction(); // Function prototype
7

```



```

8  int main()
9  {
10     int num = 1; // Local variable
11
12     cout << "In main, num is " << num << endl;
13     anotherFunction();
14     cout << "Back in main, num is " << num << endl;
15     return 0;
16 }
17
18 //*****
19 // Definition of anotherFunction
20 // It has a local variable, num, whose initial value
21 // is displayed.
22 //*****
23
24 void anotherFunction()
25 {
26     int num = 20; // Local variable
27
28     cout << "In anotherFunction, num is " << num << endl;
29 }

```

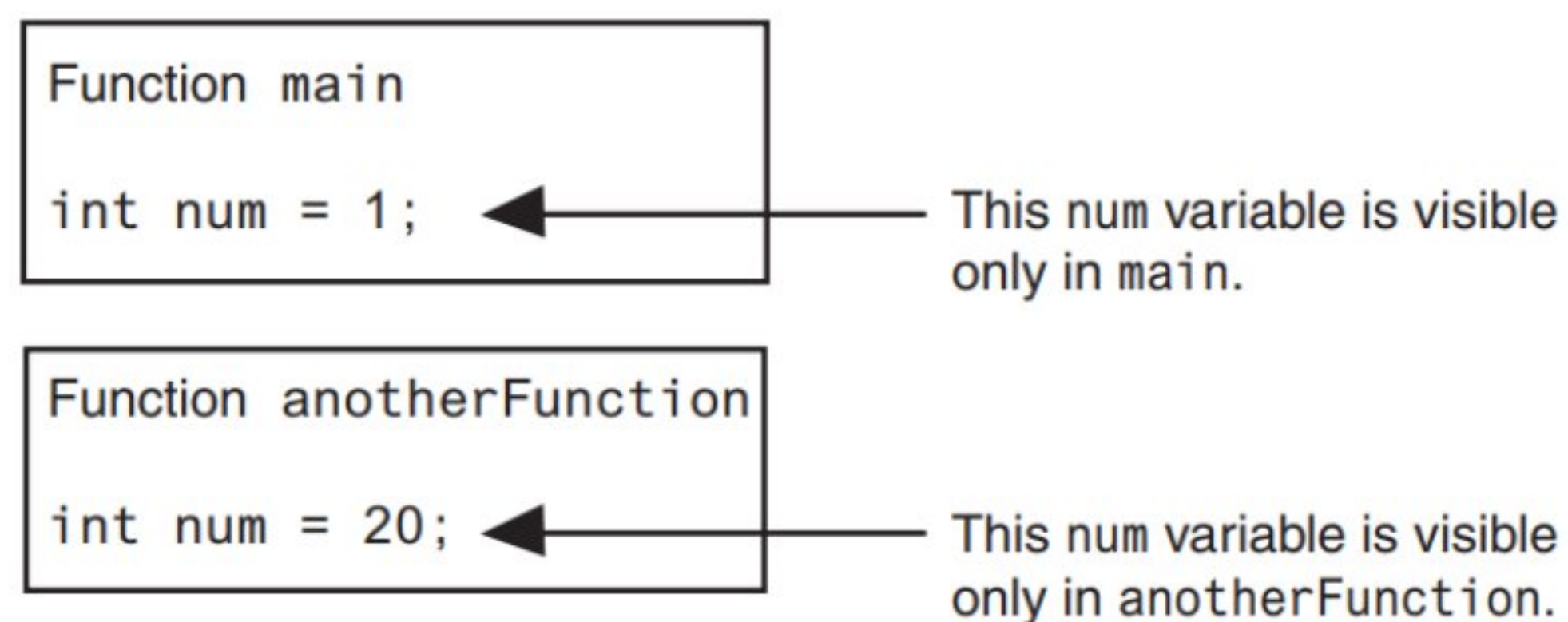
Program Output

```

In main, num is 1
In anotherFunction, num is 20
Back in main, num is 1

```

Figure 6-14 Visibility of local variables



Local Variable Lifetime

توجد المتغيرات المحلية للدالة فقط أثناء تنفيذها. يُعرف هذا بفترة حياة المتغير المحلي. عند بدء الدالة، تُنشأ متغيراتها المحلية ومتغيرات معاملاتها في الذاكرة، وعند انتهاء الدالة، تُدمر هذه المتغيرات. هذا يعني أن أي قيمة مخزنة في متغير محلي تُفقد بين استدعاءات الدالة التي يتم فيها التصريح عن المتغير.

Initializing Local Variables with Parameter Values

من الممكن استخدام متغير معامل لتهيئة متغير محلي. أحياناً يُبسّط هذا الكود في الدالة. على سبيل المثال، تذكر الإصدار الأول من دالة الجمع التي ناقشناها سابقاً:

```
int sum (int num1, int num2)
{
    int result;
    result = num1 + num2;
    return result;
}
```

في نص الدالة، يُعرّف متغير النتيجة، ثم تُعيّن جملة تعيين منفصلة $num1 + num2$ إلى النتيجة. يمكننا دمج هاتين الجملتين في جملة واحدة، كما هو موضح في النسخة المُعدّلة التالية من الدالة.

```
int sum (int num1, int num2)
{
    int result = num1 + num2;
    return result;
}
```

Global Variables

المتغير العام هو أي متغير مُعرّف خارج جميع الدوال في البرنامج. نطاق المتغير العام هو جزء البرنامج من تعريف المتغير إلى نهايته. هذا يعني أنه يمكن الوصول إلى المتغير العام بواسطة جميع الدوال المُعرّفة بعد تعريفه. يُظهر البرنامج 6-17 دالتين، `main` و `anotherFunction`، تصلان إلى نفس المتغير العام، `num`.

Program 6-17

```
1 // This program shows that a global variable is visible
2 // to all the functions that appear in a program after
3 // the variable's declaration.
4 #include <iostream>
5 using namespace std;
6
7 void anotherFunction(); // Function prototype
8 int num = 2;           // Global variable
9
10 int main()
```



```

11 {
12     cout << "In main, num is " << num << endl;
13     anotherFunction();
14     cout << "Back in main, num is " << num << endl;
15     return 0;
16 }
17
18 //*****
19 // Definition of anotherFunction *
20 // This function changes the value of the *
21 // global variable num. *
22 //*****
23
24 void anotherFunction()
25 {
26     cout << "In anotherFunction, num is " << num << endl;
27     num = 50;
28     cout << "But, it is now changed to " << num << endl;
29 }

```

Program Output

```

In main, num is 2
In anotherFunction, num is 2
But, it is now changed to 50
Back in main, num is 50

```

في البرنامج 6-17، يُعرّف المتغير `num` خارج نطاق جميع الدوال. ولأن تعريفه يظهر قبل تعريف `main` و `otherFunction`، فإن كلتا الدالتين تتمتعان بإمكانية الوصول إليه.

ملاحظة: ما لم تُهيئ المتغيرات العامة الرقمية صراحةً، فسيتم تهيئتها تلقائياً إلى الصفر. أما متغيرات الأحرف العامة، فسيتم تهيئتها إلى `NULL` (The NULL character is stored as ASCII) `0`. لا يُعيّن المتغير `globalNum` في البرنامج 6-18 إلى أي قيمة بواسطة جملة، ولكن لأنه عام، فسيتم تهيئته تلقائياً إلى الصفر.

Program 6-18

```

1 // This program has an uninitialized global variable.
2 #include <iostream>
3 using namespace std;
4
5 int globalNum; // Global variable, automatically set to zero
6
7 int main()
8 {
9     cout << "globalNum is " << globalNum << endl;
10    return 0;
11 }

```

Program Output

```

globalNum is 0

```


الآن وقد تعرفت على أساسيات المتغيرات العامة، يجب أن أنبهك إلى ضرورة تقييد استخدامها. عندما يبدأ الطلاب المبتدئون في تعلم كتابة برامج متعددة الدوال، يميلون أحياناً إلى جعل جميع متغيراتهم عامة. ويرجع ذلك عادةً إلى إمكانية الوصول إلى المتغيرات العامة بواسطة أي دالة في البرنامج دون تمريرها كوسائط. على الرغم من أن هذا النهج قد يُسهّل إنشاء البرنامج، إلا أنه عادةً ما يُسبب مشاكل لاحقاً. والأسباب هي كما يلي:

- تُصعّب المتغيرات العامة تصحيح الأخطاء. يمكن لأي جملة في البرنامج تغيير قيمة متغير عام. إذا وجدت أن القيمة الخاطئة مُخزّنة في متغير عام، فعليك تتبع كل جملة تصل إليه لتحديد مصدر القيمة الخاطئة. في برنامج يحتوي على آلاف أسطر التعليمات البرمجية، قد يكون هذا صعباً.

- عادةً ما تعتمد الدوال التي تستخدم المتغيرات العامة على هذه المتغيرات. إذا كنت ترغب في استخدام مثل هذه الدالة في برنامج مختلف، فغالباً ما ستحتاج إلى إعادة تصميمها بحيث لا تعتمد على المتغير العام.

- تُصعّب المتغيرات العامة فهم البرنامج. يمكن تعديل المتغير العام بأي جملة في البرنامج. لفهم أي جزء من البرنامج يستخدم متغيراً عاماً، يجب أن تكون على دراية بجميع أجزاء البرنامج الأخرى التي تستخدم هذا المتغير.

ولهذا السبب، يجب عدم استخدام المتغيرات العامة للأغراض التقليدية لتخزين البيانات ومعالجتها واسترجاعها. في معظم الحالات، يجب عليك إعلان المتغيرات محلياً وتمريرها كوسائط إلى الدوال التي تحتاج إلى الوصول إليها.

Global Constants

مع أنه يُنصح بتجنب استخدام المتغيرات العامة، إلا أنه يُسمح عموماً باستخدام الثوابت العامة في البرامج. الثابت العام هو ثابت مُسمّى متاح لكل دالة في البرنامج. ونظراً لاستحالة تغيير قيمة الثابت العام أثناء تنفيذ البرنامج، فلا داعي للقلق بشأن المخاطر المحتملة المرتبطة باستخدام المتغيرات العامة.

يعرض البرنامج 6-19 مثالاً على كيفية استخدام الثوابت العالمية. يحسب البرنامج الأجر الإجمالي للموظف، بما في ذلك ساعات العمل الإضافية. بالإضافة إلى الدالة main، يحتوي هذا البرنامج على دالتين: `getBasePay` و `getOvertimePay`. تقبل دالة `getBasePay` عدد ساعات العمل وتُرجع مبلغ الأجر للساعات غير الإضافية. تقبل دالة `getOvertimePay` عدد ساعات العمل وتُرجع مبلغ الأجر للساعات الإضافية، إن وُجد.

Program 6-19

```
1 // This program calculates gross pay.
2 #include <iostream>
3 #include <iomanip>
```



```

4  using namespace std;
5
6  // Global constants
7  const double PAY_RATE = 22.55;    // Hourly pay rate
8  const double BASE_HOURS = 40.0;   // Max non-overtime hours
9  const double OT_MULTIPLIER = 1.5; // Overtime multiplier
10
11 // Function prototypes
12 double getBasePay(double);
13 double getOvertimePay(double);
14
15 int main()
16 {
17     double hours,           // Hours worked
18           basePay,          // Base pay
19           overtime = 0.0,   // Overtime pay
20           totalPay;         // Total pay
21
22     // Get the number of hours worked.
23     cout << "How many hours did you work? ";
24     cin >> hours;
25
26     // Get the amount of base pay.
27     basePay = getBasePay(hours);
28
29     // Get overtime pay, if any.
30     if (hours > BASE_HOURS)
31         overtime = getOvertimePay(hours);
32
33     // Calculate the total pay.
34     totalPay = basePay + overtime;
35
36     // Set up numeric output formatting.
37     cout << setprecision(2) << fixed << showpoint;
38
39     // Display the pay.
40     cout << "Base pay: $" << basePay << endl
41           << "Overtime pay $" << overtime << endl
42           << "Total pay $" << totalPay << endl;
43     return 0;
44 }
45
46 //*****
47 // The getBasePay function accepts the number of *
48 // hours worked as an argument and returns the *
49 // employee's pay for non-overtime hours.      *
50 //*****
51
52 double getBasePay(double hoursWorked)
53 {
54     double basePay; // To hold base pay
55

```


Program 6-19

(continued)

```
56 // Determine base pay.
57 if (hoursWorked > BASE_HOURS)
58     basePay = BASE_HOURS * PAY_RATE;
59 else
60     basePay = hoursWorked * PAY_RATE;
61
62 return basePay;
63 }
64
65 /*******
66 // The getOvertimePay function accepts the number *
67 // of hours worked as an argument and returns the *
68 // employee's overtime pay. *
69 /*******
70
71 double getOvertimePay(double hoursWorked)
72 {
73     double overtimePay; // To hold overtime pay
74
75     // Determine overtime pay.
76     if (hoursWorked > BASE_HOURS)
77     {
78         overtimePay = (hoursWorked - BASE_HOURS) *
79             PAY_RATE * OT_MULTIPLIER;
80     }
81     else
82         overtimePay = 0.0;
83
84     return overtimePay;
85 }
```

Program Output with Example Input Shown in Bold

```
How many hours did you work? 48 
Base pay: $902.00
Overtime pay: $270.60
Total pay: $1172.60
```

لنلق نظرة فاحصة على البرنامج. ثلاثة ثوابت عامة مُعرّفة في الأسطر 7 و 8 و 9. يُعيّن ثابت PAY_RATE على معدل أجر الموظف بالساعة، وهو 22.55. ويُعيّن ثابت BASE_HOURS على 40، وهو عدد الساعات التي يمكن للموظف العمل بها أسبوعياً دون تقاضي أجر إضافي. ويُعيّن ثابت OT_MULTIPLIER على 1.5، وهو مُضاعف معدل الأجر لساعات العمل الإضافية. هذا يعني أن معدل أجر الموظف بالساعة يُضرب في 1.5 لجميع ساعات العمل الإضافية. ولأن هذه الثوابت عامة ومُعرّفة قبل جميع الدوال في البرنامج، يُمكن لجميع الدوال الوصول إليها. على سبيل المثال، تصل دالة getBasePay إلى ثابت BASE_HOURS في السطرين 57 و 58، وثابت PAY_RATE في السطرين 58 و 60. تصل دالة getOvertimePay إلى ثابت

BASE_HOURS في السطرين 76 و78، وثابت PAY_RATE في السطر 79، وثابت OT_MULTIPLIER في السطر 79.

Local and Global Variables with the Same Name

لا يُمكن استخدام متغيرين محليين بنفس الاسم في نفس الدالة. وينطبق هذا أيضًا على متغيرات المعاملات. متغير المعاملات هو، في جوهره، متغير محلي. لذا، لا يُمكنك إعطاء اسم مُتغير معاملات ومتغير محلي في نفس الدالة.

ومع ذلك، يُمكنك استخدام مُتغير محلي أو مُتغير معاملات بنفس اسم مُتغير عام، أو ثابت عام. عند القيام بذلك، يُخفي اسم المُتغير المحلي أو مُتغير المعاملات اسم المُتغير العام أو الثابت العام. هذا يعني أن اسم المُتغير العام أو الثابت العام يكون مخفيًا باسم المُتغير المحلي أو مُتغير المعاملات.

على سبيل المثال، انظر إلى البرنامج 6-20. يحتوي هذا البرنامج على ثابت عام يُسمى BIRDS، مضبوط على 500. تحتوي دالة كاليفورنيا على ثابت محلي يُسمى BIRDS، مضبوط على 10000.

Program 6-20

```
1 // This program demonstrates how a local variable
2 // can shadow the name of a global constant.
3 #include <iostream>
4 using namespace std;
5
6 // Global constant.
7 const int BIRDS = 500;
8
9 // Function prototype
10 void california();
11
12 int main()
13 {
14     cout << "In main there are " << BIRDS
15         << " birds.\n";
16     california();
17     return 0;
18 }
19
20 //*****
21 // california function *
22 //*****
23
24 void california()
25 {
26     const int BIRDS = 10000;
27     cout << "In california there are " << BIRDS
28         << " birds.\n";
29 }
```

Program Output

```
In main there are 500 birds.
In california there are 10000 birds.
```


6.11 Static Local Variables

إذا استدعيت دالة أكثر من مرة في برنامج، فإن القيم المخزنة في متغيراتها المحلية لا تبقى بين استدعاءات الدالة. وذلك لأن هذه المتغيرات تُدمر عند انتهاء الدالة، ثم تُنشأ مجددًا عند بدء تشغيلها. هذا موضح في البرنامج 6-21.

Program 6-21

```
1 // This program shows that local variables do not retain
2 // their values between function calls.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototype
7 void showLocal();
8
9 int main()
10 {
11     showLocal();
12     showLocal();
13     return 0;
14 }
15
16 //*****
17 // Definition of function showLocal. *
18 // The initial value of localNum, which is 5, is displayed. *
19 // The value of localNum is then changed to 99 before the *
20 // function returns. *
21 //*****
22
23 void showLocal()
24 {
25     int localNum = 5; // Local variable
26
27     cout << "localNum is " << localNum << endl;
28     localNum = 99;
29 }
```

Program Output

```
localNum is 5
localNum is 5
```


أحياناً يكون من المستحسن أن يتذكر البرنامج القيمة المخزنة في متغير محلي بين استدعاءات الدالة. يمكن تحقيق ذلك بجعل المتغير ثابتاً. لا تُدمر المتغيرات المحلية الثابتة عند عودة الدالة، بل تبقى موجودة طوال عمر البرنامج، حتى لو اقتصر نطاقها على الدالة التي عُرِّفت فيها فقط.

يوضح البرنامج 6-22 بعض خصائص المتغيرات المحلية الثابتة:

Program 6-22

```
1 // This program uses a static local variable.
2 #include <iostream>
3 using namespace std;
4
5 void showStatic(); // Function prototype
6
7 int main()
8 {
9     // Call the showStatic function five times.
10    for (int count = 0; count < 5; count++)
11        showStatic();
12    return 0;
13 }
14
15 //*****
16 // Definition of function showStatic. *
17 // statNum is a static local variable. Its value is displayed *
18 // and then incremented just before the function returns. *
19 //*****
20
21 void showStatic()
22 {
23     static int statNum;
24
25     cout << "statNum is " << statNum << endl;
26     statNum++;
27 }
```

Program Output

```
statNum is 0
statNum is 1
statNum is 2
statNum is 3
statNum is 4
```

في السطر 26 من البرنامج 6-22، تتم زيادة قيمة statNum في دالة showStatic، وتحتفظ بقيمتها بين كل استدعاء للدالة. لاحظ أنه على الرغم من عدم تهيئة statNum صراحةً، إلا أنها تبدأ من الصفر. وكما هو الحال مع المتغيرات العامة، تُهيأ جميع المتغيرات المحلية الثابتة إلى الصفر افتراضياً. (بالطبع، يمكنك تحديد قيمة تهيئة خاصة بك، إذا لزم الأمر).

إذا حددت قيمة تهيئة لمتغير محلي ثابت، فإن التهيئة تحدث مرة واحدة فقط. وذلك لأن التهيئة تحدث عادةً عند إنشاء المتغير، بينما تُنشأ المتغيرات المحلية الثابتة مرة واحدة فقط أثناء تشغيل البرنامج. يوضح البرنامج 6-23، وهو تعديل طفيف للبرنامج 6-22، هذه النقطة.

Program 6-23

```
1 // This program shows that a static local variable is only
2 // initialized once.
3 #include <iostream>
4 using namespace std;
5
6 void showStatic(); // Function prototype
7
8 int main()
9 {
10     // Call the showStatic function five times.
11     for (int count = 0; count < 5; count++)
12         showStatic();
13     return 0;
14 }
15
16 //*****
17 // Definition of function showStatic. *
18 // statNum is a static local variable. Its value is displayed *
19 // and then incremented just before the function returns. *
20 //*****
21
22 void showStatic()
23 {
24     static int statNum = 5;
25
26     cout << "statNum is " << statNum << endl;
27     statNum++;
28 }
```

Program Output

```
statNum is 5
statNum is 6
statNum is 7
statNum is 8
statNum is 9
```


6.12 Default Arguments

المفهوم: تُمرّر الوسائط الافتراضية إلى المَعْلَمَات تلقائيًا في حال عدم وجود أي وسيطة في استدعاء الدالة.

من الممكن تعيين وسائط افتراضية (*default arguments*) لمَعْلَمَات الدالة. تُمرّر وسيطة افتراضية إلى المَعْلَمَة عند حذف الوسيطة الفعلية من استدعاء الدالة. عادةً ما تُدرج الوسائط الافتراضية في النموذج الأولي للدالة. إليك مثال:

```
void showArea (double = 20.0, double = 10.0);
```

الوسائط الافتراضية هي قيم حرفية أو ثوابت يسبقها عامل =، وتظهر بعد أنواع البيانات المدرجة في نموذج أولي للدالة. بما أن أسماء المَعْلَمَات اختيارية في نماذج الدوال الأولية، يمكن أيضًا إعلان نموذج المثال كـ:

```
void showArea (double length = 20.0, double width = 10.0);
```

في كلا النموذجين الأوليين، تحتوي دالة `showArea` على معامليين مزدوجين. الأول مُخصص له الوسيطة الافتراضية 20.0، والثاني مُخصص له الوسيطة الافتراضية 10.0. فيما يلي تعريف الدالة:

```
void showArea (double length, double width)  
{  
    double area = length * width;  
    cout << "The area is " << area << endl;  
}
```

الوسيطة الافتراضية للطول هي 20.0، والوسيطة الافتراضية للعرض هي 10.0. نظرًا لامتلاك كلا المعامليين لوسيطتين افتراضيتين، يُمكن حذفهما اختياريًا عند استدعاء الدالة، كما هو موضح هنا:

```
showArea ();
```

في استدعاء هذه الدالة، سيتم تمرير كلا الوسيطتين الافتراضيتين إلى المَعْلَمَات. مَعْلَمَة الطول ستكون قيمتها 20.0، ومَعْلَمَة العرض ستكون قيمتها 10.0. سيكون ناتج الدالة:

```
The area is 200
```

تُستخدم الوسائط الافتراضية فقط عند حذف الوسائط الفعلية من استدعاء الدالة. في الاستدعاء التالي، يتم تحديد الوسيطة الأولى، ولكن يتم حذف الثانية:

```
showArea (12.0);
```


سيتم تمرير القيمة 12.0 إلى الطول، بينما سيتم تمرير القيمة الافتراضية 10.0 إلى العرض. سيكون ناتج الدالة:

The area is 120

بالطبع، يمكن تجاوز جميع الوسائط الافتراضية. في استدعاء الدالة أدناه، يتم توفير الوسائط لكلا المعاملين:

showArea (12.0, 5.5);

سيكون مخرج استدعاء الوظيفة أعلاه:

The area is 66

تحذير! يجب تعيين الوسيطات الافتراضية للدالة في أول ظهور لاسم الدالة. عادةً ما يكون هذا هو النموذج الأولي للدالة (function prototype).

يستخدم البرنامج 6-24 دالة لعرض النجوم على الشاشة. تُمرّر إلى الدالة وسيطات تُحدّد عدد أعمدة وصفوف النجوم المراد عرضها. تُقدّم وسيطات افتراضية لعرض صف واحد من 10 نجوم.

Program 6-24

```
1 // This program demonstrates default function arguments.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype with default arguments
6 void displayStars(int = 10, int = 1);
7
8 int main()
9 {
10     displayStars();           // Use default values for cols and rows.
11     cout << endl;
12     displayStars(5);         // Use default value for rows.
13     cout << endl;
14     displayStars(7, 3);      // Use 7 for cols and 3 for rows.
15     return 0;
16 }
17
18 //*****
19 // Definition of function displayStars.
20 // The default argument for cols is 10 and for rows is 1.
21 // This function displays a square made of asterisks.
22 //*****
23
24 void displayStars(int cols, int rows)
25 {
26     // Nested loop. The outer loop controls the rows
27     // and the inner loop controls the columns.
28     for (int down = 0; down < rows; down++)
29     {
30         for (int across = 0; across < cols; across++)
31             cout << " ";
32         cout << endl;
33     }
34 }
```

Program Output

```
*****
*****
*****
*****
*****
```


6.13 Using Reference Variables as Parameters

المفهوم: عند استخدامها كمعاملات، تسمح متغيرات المرجع للدالة بالوصول إلى وسيطة المعامل الأصلية. كما تُجرى تغييرات على الوسيطة.

توفر لغة C++ نوعًا خاصًا من المتغيرات يُسمى متغير مرجعي (reference variable)، والذي يسمح، عند استخدامه كمعامل دالة، بالوصول إلى الوسيطة الأصلية.

متغير المرجع هو اسم مستعار (alias) لمتغير آخر. أي تغييرات تُجرى على متغير المرجع تُجرى فعليًا على المتغير الذي يُمثله. باستخدام متغير مرجعي كمعامل، يمكن للدالة تغيير متغير مُعرّف في دالة أخرى.

تُعرّف المتغيرات المرجعية مثل المتغيرات العادية، باستثناء وضع علامة (&) قبل الاسم. على سبيل المثال، يُحوّل تعريف الدالة التالي المعامل refVar إلى متغير مرجعي:

```
void doubleNum (int &refVar)
{
    refVar *= 2;
}
```

ملاحظة: يسمى المتغير refVar "مرجعًا إلى int".

تضاعف هذه الدالة refVar بضربه في 2. ولأن refVar متغير مرجعي، فإن هذا الإجراء يُنفَّذ فعليًا على المتغير الذي مُرّر إلى الدالة كمعامل. عند إنشاء نموذج أولي لدالة بمتغير مرجعي، تأكد من تضمين علامة العطف (&) بعد نوع البيانات. إليك النموذج الأولي لدالة doubleNum:

```
void doubleNum (int &);
```



NOTE: Some programmers prefer not to put a space between the data type and the ampersand. The following prototype is equivalent to the one above:

```
void doubleNum(int&);
```



NOTE: The ampersand must appear in both the prototype and the header of any function that uses a reference variable as a parameter. It does not appear in the function call.

يوضح البرنامج 6-25 كيفية عمل الدالة doubleNum.

Program 6-25

```
1 // This program uses a reference variable as a function
2 // parameter.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototype. The parameter is a reference variable.
7 void doubleNum(int &);
```


Program 6-25*(continued)*

```

8
9  int main()
10 {
11     int value = 4;
12
13     cout << "In main, value is " << value << endl;
14     cout << "Now calling doubleNum..." << endl;
15     doubleNum(value);
16     cout << "Now back in main. value is " << value << endl;
17     return 0;
18 }
19
20 //*****
21 // Definition of doubleNum.
22 // The parameter refVar is a reference variable. The value
23 // in refVar is doubled.
24 //*****
25
26 void doubleNum (int &refVar)
27 {
28     refVar *= 2;
29 }

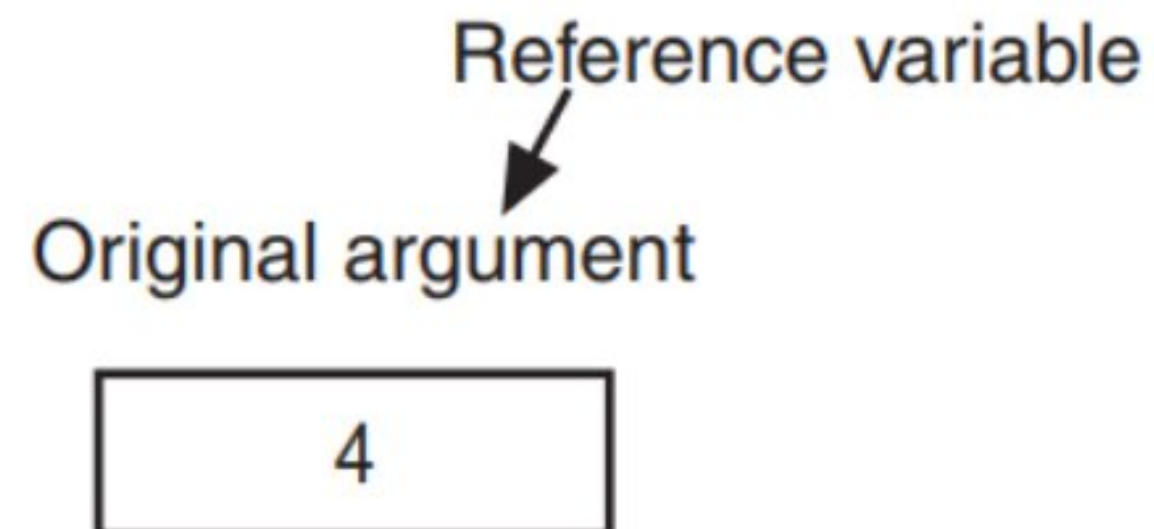
```

Program Output

```

In main, value is 4
Now calling doubleNum...
Now back in main. value is 8

```

Figure 6-15 Reference variable

تذكر أن وسيطات الدالة تُمرَّر عادةً بالقيمة، أي تُمرَّر نسخة من قيمة الوسيطة إلى متغير المعامل. عند استخدام معامل مرجعي، يُقال إن الوسيطة تُمرَّر بالمرجع.

البرنامج 26-6 هو تعديل للبرنامج 25-6. تمت إضافة الدالة `getNum`.

تطلب هذه الدالة من المستخدم إدخال رقم، يُخزَّن في `userNum`. `userNum` هو مرجع لقيمة متغير `main`.

Program 6-26

```
1 // This program uses reference variables as function parameters.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototypes. Both functions use reference variables
6 // as parameters.
7 void doubleNum(int &);
8 void getNum(int &);
9
10 int main()
11 {
12     int value;
13
14     // Get a number and store it in value.
15     getNum(value);
16
17     // Double the number stored in value.
18     doubleNum(value);
19
20     // Display the resulting number.
21     cout << "That value doubled is " << value << endl;
22     return 0;
23 }
24
25 //*****
26 // Definition of getNum. *
27 // The parameter userNum is a reference variable. The user is *
28 // asked to enter a number, which is stored in userNum. *
29 //*****
30
31 void getNum(int &userNum)
32 {
33     cout << "Enter a number: ";
34     cin >> userNum;
35 }
36
37 //*****
38 // Definition of doubleNum. *
39 // The parameter refVar is a reference variable. The value *
40 // in refVar is doubled. *
41 //*****
42
43 void doubleNum (int &refVar)
44 {
45     refVar *= 2;
46 }
```

Program Output with Example Input Shown in Bold

```
Enter a number: 12 
That value doubled is 24
```



NOTE: Only variables may be passed by reference. If you attempt to pass a nonvariable argument, such as a literal, a constant, or an expression, into a reference parameter, an error will result. Using the `doubleNum` function as an example, the following statements will cause an error:

```
doubleNum(5);           // Error
doubleNum(userNum + 10); // Error
```


6.14 Overloading Functions

المفهوم: قد تحمل دالتان أو أكثر الاسم نفسه، طالما أن قوائم معاملاتها مختلفة.

في بعض الأحيان، سننشئ دالتين أو أكثر نُجريان العملية نفسها، ولكن باستخدام مجموعة مختلفة من المعاملات أو معاملات من أنواع بيانات مختلفة. على سبيل المثال، في البرنامج 6-13، توجد دالة مربعة (square) تستخدم معاملاً مزدوجاً (double). ولكن، لنفترض أنك تريد أيضاً دالة مربعة تعمل حصرياً مع الأعداد الصحيحة، وتقبل عدداً صحيحاً كمعامل لها. ستؤدي كلتا الدالتين نفس الوظيفة: إرجاع مربع معاملهما.

الفرق الوحيد هو نوع البيانات المُستخدم في العملية. إذا كنت ستستخدم هاتين الدالتين في نفس البرنامج، فيمكنك تعيين اسم فريد لكل دالة. على سبيل المثال، قد يُطلق على الدالة التي تُربع عدداً صحيحاً اسم squareInt، وعلى الدالة التي تُربع عدداً مزدوجاً اسم squareDouble. مع ذلك، تسمح لك لغة C++ بزيادة تحميل الدوال. هذا يعني أنه يمكنك تعيين الاسم نفسه لوظائف متعددة، طالما أن قوائم معلماتها مختلفة. يستخدم البرنامج 6-28 دالتين مربعيتين محملتين.

Program 6-28

```
1 // This program uses overloaded functions.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 // Function prototypes
7 int square(int);
8 double square(double);
9
10 int main()
11 {
12     int userInt;
13     double userFloat;
14
15     // Get an int and a double.
16     cout << fixed << showpoint << setprecision(2);
17     cout << "Enter an integer and a floating-point value: ";
18     cin >> userInt >> userFloat;
19
20     // Display their squares.
21     cout << "Here are their squares: ";
22     cout << square(userInt) << " and " << square(userFloat);
23     return 0;
24 }
25
26 /*******
27 // Definition of overloaded function square.
28 // This function uses an int parameter, number. It returns the
29 // square of number as an int.
30 /*******
```


Program 6-28 (continued)

```

31
32 int square(int number)
33 {
34     return number * number;
35 }
36
37 //*****
38 // Definition of overloaded function square.          *
39 // This function uses a double parameter, number. It returns *
40 // the square of number as a double.                  *
41 //*****
42
43 double square(double number)
44 {
45     return number * number;
46 }

```

Program Output with Example Input Shown in Bold

Enter an integer and a floating-point value: **12 4.2**
 Here are their squares: 144 and 17.64

في لغة C++، لكل دالة توقيعها الخاص (function signature). توقيع الدالة هو اسمها وأنواع بيانات معاملاتها بالترتيب الصحيح. دوال المربع في البرنامج 6-28 تحمل التوقيعات التالية:

square(int)

square(double)

عند استدعاء دالة مُحَمَّلة، تستخدم لغة C++ توقيع الدالة لتمييزها عن الدوال الأخرى التي تحمل الاسم نفسه. في البرنامج 6-28، عند تمرير وسيطة int إلى square، تُستدعى نسخة الدالة التي تحتوي على مُعامل int. وبالمثل، عند تمرير وسيطة double إلى square، تُستدعى النسخة التي تحتوي على مُعامل double.

6.15 The exit() Function

المفهوم: تؤدي دالة exit() إلى إنهاء البرنامج، بغض النظر عن الدالة أو آلية التحكم المُنفَّذة.

يتوقف برنامج C++ عن التنفيذ عند مواجهة عبارة return في الدالة main. مع ذلك، لا يتوقف البرنامج عند انتهاء دوال أخرى. يعود التحكم في البرنامج إلى المكان الذي يلي استدعاء الدالة مباشرةً. في بعض الأحيان، تُستلزم ظروف نادرة إنهاء البرنامج في دالة أخرى غير main. لتحقيق ذلك، تُستخدم دالة exit.

عند استدعاء دالة exit، فإنها تؤدي إلى إيقاف البرنامج، بغض النظر عن الدالة التي استدُعيت. يوضح البرنامج 6-30 استخدامها.

Program 6-30

```
1 // This program shows how the exit function causes a program
2 // to stop executing.
3 #include <iostream>
4 #include <cstdlib> // Needed for the exit function
5 using namespace std;
6
7 void function(); // Function prototype
8
9 int main()
10 {
11     function();
12     return 0;
13 }
14
15 /*******
16 // This function simply demonstrates that exit can be used *
17 // to terminate a program from a function other than main. *
18 /*******
19
20 void function()
21 {
22     cout << "This program terminates with the exit function.\n";
23     cout << "Bye!\n";
24     exit(0);
25     cout << "This message will never be displayed\n";
26     cout << "because the program has already terminated.\n";
27 }
```

Program Output

```
This program terminates with the exit function.
Bye!
```

لاستخدام دالة الخروج (exit)، يجب تضمين ملف رأس <cstdlib>. لاحظ أن الدالة تأخذ مُعاملًا صحيحًا. هذا المُعامل هو رمز الخروج الذي ترغب في أن يُعيده البرنامج إلى نظام تشغيل الحاسوب. يُستخدم هذا الرمز أحيانًا خارج البرنامج للإشارة إلى ما إذا كان البرنامج قد انتهى بنجاح أو نتيجة عطل. في البرنامج 6-30، يتم تمرير رمز الخروج صفر، والذي يُشير عادةً إلى خروج ناجح. إذا لم تكن متأكدًا من الرمز المُستخدم مع دالة الخروج، فهناك ثابتان مُسميان، هما EXIT_FAILURE وEXIT_SUCCESS، مُعرّفات في <cstdlib> لاستخدامهما. يُعرّف الثابت EXIT_FAILURE بأنه رمز الإنهاء الذي يُمثل عادةً خروجًا غير ناجح في نظام التشغيل الحالي. إليك مثال على استخدامه:

exit(EXIT_FAILURE);

يُعرّف الثابت EXIT_SUCCESS بأنه رمز الإنهاء الذي يُمثل عادةً خروجًا ناجحًا في نظام التشغيل الحالي. فيما يلي مثال:

exit(EXIT_SUCCESS);

انتهت المحاضرة السادسة